AVEVA

CONTINUAL PROGRESSION

AVEVA Plant (12 Series)

Programmable Macro Language: Form Design

TRAINING GUIDE

TM-1402

Revision Log

Date	Revision	Description of Revision	Author	Reviewed	Approved
15/01/2010	0.1	PML Training Manuals Split. Issued for Review	JW		
27/01/2010	0.2	Reviewed	JW	EJW	
20/05/2010	1.0	Issued for Training 12.0.SP5	JW	EJW	RP

Updates

All headings containing updated or new material will be highlighted.

Suggestion / Problems

If you have a suggestion about this manual or the system to which it refers please report it to the AVEVA Group Solutions Centre at **gsc@aveva.com**

This manual provides documentation relating to products to which you may not have access or which may not be licensed to you. For further information on which products are licensed to you please refer to your licence conditions.

Visit our website at **http://www.aveva.com**

Disclaimer

Trademarks

Copyright

Printed by AVEVA Solutions on 26 May 2010

AVEVA Solutions Ltd, High Cross, Madingley Road, Cambridge, CB3 0HB, United Kingdom.

Contents

CHAPTER 1

1 Introduction

This manual is designed to give an introduction to the AVEVA Plant Programming Macro Language. There is no intention to teach software programming but only provide instruction on how to customise PDMS using Programmable Macro Language (PML) in AVEVA Plant.

This training guide is supported by the reference manuals available within the products installation folder. References will be made to these manuals throughout the guide.

1.1 Aim

The following points need to be understood by the trainees:

- Understand how PML can be used to customise AVEVA Plant
- Understand how to create Forms
- Understand how to use the built-in objects of AVEVA Plant
- Understand the use of Addins to customise the environment.

1.2 Objectives

At the end of this training, you will have:

- Knowledge of how Forms are created and how Form Gadgets and Form Members are defined
- Understanding of Menus and Toolbars are defined with PML
- Understanding of Collections, Basic Event Driven Graphics, Error Tracing and PML Encryption

1.3 Prerequisites

The participants must have:

- Completed an AVEVA Basic Design Course and have a familiarity with PDMS
- Completed the PML: Macros & Functions Course or have familiarity with PML.

1.4 Course Structure

Training will consist of oral and visual presentations, demonstrations, worked examples and set exercises. Each trainee will be provided with some example files to support this guide. Each workstation will have a training project, populated with model objects. This will be used by the trainees to practice their methods, and complete the set exercises.

1.5 Using this Guide

Certain text styles are used to indicate special situations throughout this document, here is a summary;

Menu pull downs and button press actions are indicated by **bold dark turquoise text**.

Information the user has to Key-in **'will be red and BOLD'**

Additional information will be highlighted

Reference to other documentation will be separate

System prompts should be bold and italic in inverted commas i.e. ***'Choose function'***

Example files or inputs will be in the `courier new font` with colours and styles used as before.

2 Forms

2.1 Forms are Global Objects

Forms are objects which are represented by **GLOBAL VARIABLES**. The gadgets on a form are objects that are **MEMBERS** of the form object. As it is a global variable, once defined, the information held within a form is available to the rest of the program.

To find out information about the form, it can be queried as if it was an object. For example, show the Graphics Settings form (**Settings>Graphics…**)

This form could also have been shown using **`show !!gphsettings`**. The **SHOW** syntax loads and displays a form. If you wish to load the form, but not see it, you could type **`load !!gphsettings`**

Type **`q var !!gphsettings`** onto the command line. The information is a list of the members of the form. Compare this list against some of the gadgets on the form. As these gadgets are objects as well, we can find out further information.

The first gadget listed is called **OK** (representing the OK button on the form). To find out information about it, type **`q var !!gphsettings.ok`**. Specific information about the gadget can be queried directly e.g.:

```
q var !!gphsettings.ok.tag
q var !!gphsettings.ok.val
q var !!gphsettings.ok.active
```

ⓘ *To get the name of a shown form, type **`show !!pmlforms`**. This form can list shown forms*

2.2 Dynamic Loading of Objects, Forms and Functions

When a PML object is used for the first time, it is loaded automatically. This applies to both **FORMS** and **OBJECTS** e.g.:

```
!Person = object PRIMEMINISTER()
show !!MyInputForm
```

Once an object is loaded by PDMS, the definition is held by PDMS. This means that if the object definition is changed outside PDMS whilst it is loaded, the form will need to be reloaded. To reload a form type pml reload form or object followed by the object name e.g.:

```
pml reload form !!MyInputForm
pml reload object PRIMEMINISTER
```

If a new file is created whilst PDMS is open, the file will not be mapped (even if it is saved in an appropriate file path). The remap files type **`pml rehash`**. This will remap all the files within the first file path in the PMLLIB variable.

The remap all files in all the file paths, type **`pml rehash all`**

2.3 Defining a Form

A form is defined within a .pmlfrm file and should be saved under the **PMLLIB** searchpath. The file will define the form, its members and any associated methods. The file will first contain the form setup (gadgets, members etc) and then the methods, following the below format:

```
setup form !!exampleForm
  ...
exit

define method .init()
  ...
endmethod
```

2.3.1 Using the .net Framework

Form objects make use of the .net framework. This allows the following features to be included:

- Docking forms (automatic resizing)
- Anchoring gadgets (useful when resizing)
- Multicolumn lists
- Tabs on Forms
- Menu Additions to existing applications:
- Toolbar Additions to existing applications
- DBAR Conversions

Although it is now possible to dock forms, it does not apply to every form. There are some rules to consider when deciding if a form should dock:

- Does the form need to remain open?
- Is the form used heavily
- If a form has an OK/Cancel button, it should not need docking
- If a form has a menubar, it CANNOT be docked

To declare a form as able to dock, this has to be done on the top of the definition. By including **`dialog dock`**, we are stating that the form will have the ability to dock:

```
setup form !!exampleForm dialog dock left
```

To define a floating form that is able to dock, use the following line:

```
setup form !!exampleForm dialog resizeable
```

To define a form of a certain size in the centre of the screen, use the following line:

```
setup form !!exampleForm document at xr 0.5 yr 0.5 size 100 100
```

If no additional details are included, the default form creation is a Dialog, non-resizeable, size adjusted automatically to fit contents form. A form will always be as big as it needs to be to display the contents.

ⓘ *For more examples, refer to the FORM object in the PDMS Software Customisation Reference Manual*

2.3.2 Showing and Hiding Forms

Forms are found through the **PMLLIB** search path, there is no need to load them individually. To show a form use **`show !!formname`**. This will load the form definition and show it in one go!

Sometimes it is useful to have the form loaded with seeing it (e.g. to refer to stored information). A form can be loaded (but not shown) by typing **`loadform !!formname`**

2.3.3 Built-in Methods for Forms

Although is possible to define user-defined methods within user-defined forms (discussed later), all **FORM** objects have built-in methods available. Try on the following on the command line:

```
!!gphsettings.show()
q var !!gphsettings.shown()
!!gphsettings.hide()
q var !!gphsettings.shown()
```

In this example, the **.show()** method is used to show the form, the **.shown()** method returns whether the form is shown and the **.hide()** method hides the form.

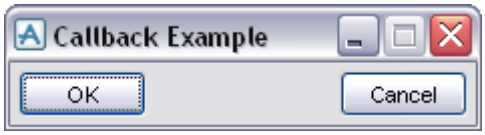
For more examples, refer to the FORM object in the PDMS Software Customisation Reference Manual

2.4 Callbacks

If an object has a **CALLBACK** member, it can be given a callback string. This means that if the user interacts with the object, an action can be performed. The callback can do one of three things (1) show a form (2) execute a command directly or (3) run a function or method

A **FORM** object has some special callbacks which are used when the form completes certain actions (e.g. is shown, is closed). The following example demonstrates the various callbacks on a **FORM** object. Show the form by typing **`show !!exampleCallback.`** Test the form by showing and closing it.

```
setup form !!exampleCallback
  !this.formTitle      = |Callback Example|
  !this.initCall       = |!this.init()|
  !this.firstShownCall = |!this.firstShown()|
  !this.okCall         = |!this.okCall()|
  !this.cancelCall     = |!this.cancelCall()|
  !this.quitCall       = |!this.quitCall()|
  !this.killingCall    = |!this.killCall()|

  button .ok     |  OK  |        OK
  button .cancel |Cancel| at x20 CANCEL
exit

define method .exampleCallback()
  $p Constructor Method
endmethod

define method .init()
  $p Initialise Method
endmethod

define method .firstShown()
  $p First Shown Method
endmethod

define method .okCall()
  $p OK Method
endmethod

define method .cancelCall()
  $p Cancel Method
endmethod

define method .quitCall()
  $p Quit Method
endmethod

define method .killCall()
  $p Kill Method
endmethod
```



The callbacks on a form object can run any command syntax, global function, local method. In this example, the callbacks call local methods of similar names. They could have called any method, even the same method. The only important part is that there is a local method of that name

Callbacks can reference any method/function/command

These event callbacks are built-in members of a **FORM** object. They are typically defined at the top of the form definition. There are 7 main event callbacks available in PDMS.

- The **CONSTRUCTOR** method was called when the form loaded (notice, it has the same name as the form).
- The **INITCALL** method is called everytime the form is shown (perfect for setting default values)
- The **FIRSTSHOWNCALL** method is called the when the form is activated (first shown)
- The **OKCALL** method is called by any button gadget with the OK in its definition
- The **CANCELCALL** method is called by any button gadget with CANCEL in its definition
- The **QUITCALL** method is called by clicking the close button on the form.
- The **KILLINGCALL** method is called when the form is unloaded (killed or reloaded)

ⓘ *If no callbacks are defined for OKCALL, CANCELCALL and QUITCALL, the default is to hide the form only*

2.5 Form Gadgets

There are many kinds of form gadgets, each an object that will have its own **MEMBERS** and **METHODS**. When you are defining gadgets on a form, there are two common aims:

- Define the area to be taken up on the form
- Define the action to be taken if the gadget is interacted with

It is the position and size of the gadget that determines the area taken up and its action is defined by its **CALLBACK** member.

2.5.1 Built-in Members and Methods for Gadgets

As gadgets are objects, there are a variety of useful members and built-in methods that can be used. Based on the previous example, type the following onto the command line:

To grey-out the OK button
```
!!exampleCallback.ok.active = FALSE
```

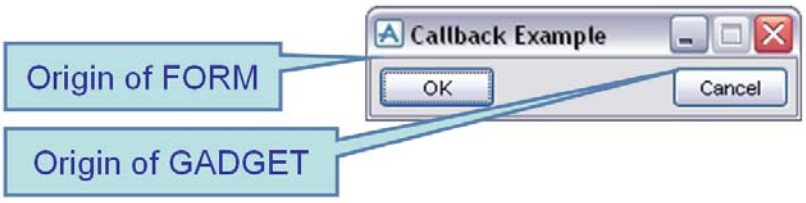
To hide the CANCEL button
```
!!exampleCallback.cancel.visible = FALSE
```

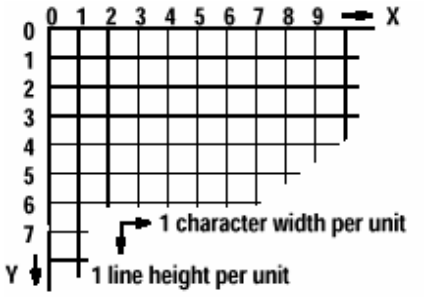
Apply a tooltip to the OK button
```
!!exampleCallback.ok.setToolTip(|This is an OK button|)
```

📖 *For further information, PDMS Software Customisation Reference Manual*

2.5.2 Gadget Positioning

Gadgets are positioned on a form from top left using the **AT** syntax. The **AT** syntax defines the origin of the gadget in relation to the owning object (i.e. FORM or FRAME). .



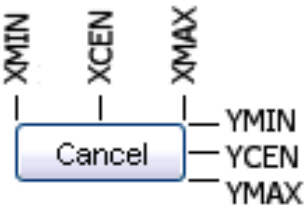


Gadgets can be positioned explicitly or in relation to other objects. When referring to other objects, there 6 known positions on a gadget: **XMIN**, **XCEN**, **XMAX**, **YMIN**, **YCEN** and **YMAX**. These refer to fixed position in the x and y directions on the referenced gadget. A gadget can be thought of as an enclosing box that will enclose the geometry of the gadget (including its name tag if specified).

To position a gadget at a known position use:

```
at x 0 y 0
```

To position the above CANCEL button using the XMAX and YMIN of OK use:




```
at xmax.ok + 10 ymin.ok
```

To position a DISMISS button in the bottom corner of a form use:

```
at xmax form – size ymax form
```

For more positioning syntax, refer to the PDMS Software Customisation Reference Manual

The available syntax and its order can be derived by referring to the **SYNTAX GRAPHS** in the reference manual. These diagrams are available for most Gadgets and can be used when initially defining them. The picture below is an example of the Syntax Graph for gadget positioning (**<fgprl>** refers to another graph):

```
>-- <fgpos> -- AT --+- val val ------------------------------.
                    +- X val ------------.                    |
                    +- XMIN -.           |                    |
                    +- XCEN -|           |                    |
                    +- XMAX -+- <fgprl> -|                    |
                    '--------'-----------+- Y val ------------|
                                         +- YMIN -.           |
                                         +- YCEN -|           |
                                         +- YMAX -'- <fgprl> -|
                                         `--------------------'-->
```

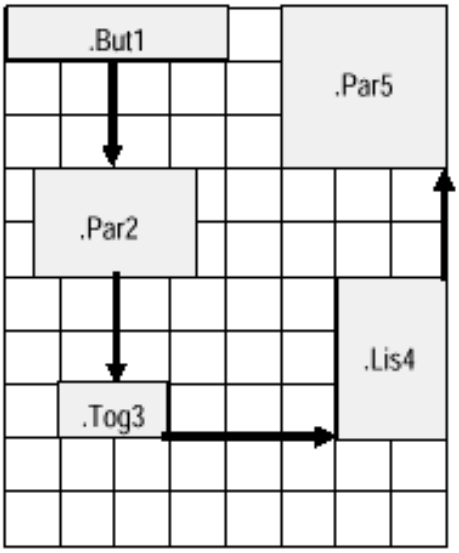
ⓘ *Refer PDMS Software Customisation Reference Manual for more guidance on these graphs*

2.5.2.1 Position Gadgets Using the Path Command

The path command can be used to define the logical position of subsequent gadgets. This method has been superseded by the previous method and has been included for information.

After a gadget has been defined, the next gadget is positioned based on a PATH, HDIST or VDIST and HALIGN or VALIGN. As an example, see the picture below:

```
Button .But1                    $* default placement
PATH down
HALIGN centre
VDIST 2

paragraph .Par2 width 3 height 2 $* auto-placed
toggle .Tog3                     $* auto-placed

PATH right
HDIST 3
VALIGN bottom

List .Lis4 width 2 height 3      $* auto-placed

PATH up
HALIGN right

Paragraph .Par5 width 3 height 3 $* auto-placed
```



2.5.3 Docking and Anchoring Gadgets

After a gadget has been loaded its size and position are locked. To modify the position or size manually, the alterations have to be made in the file and the form reloaded. As forms can be resized, it is necessary for gadgets to move/resize so the layout of the form remains.

There are two available syntax definitions that can help: **DOCK** or **ANCHOR**. This syntax should be included when defining a gadget and can be included if **<fgdock>** or **<fganch>** are included in the syntax graph of the gadget. Once a gadget is declared as Anchored or Docking it will remain so. If a change is required, the form definition should be updated and the form reloaded.

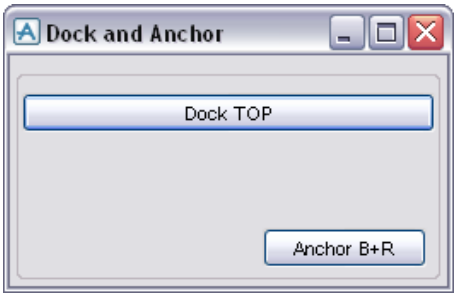
ⓘ *The DOCK and ANCHOR are mutually exclusive so only one is defined per gadget*

- **ANCHOR** controls the position of an edge of the gadget relative to the corresponding edge of its container. For example, if a DISMISS button is anchored to the Right + Bottom, it will remain the same distance from the bottom, right of the form if it is resized.
- **DOCK** forces the gadget to fill the available space in a certain direction. For example, if a list is docked to the left, it will maintain its width, but its height will change it fill its container. **DOCK FILL** is very useful for ensuring a gadget is the full size of its container.

📖 *Refer PDMS Software Customisation Reference Manual for the Syntax Graphs.*

Show the example form by typing **`show !!exampleDocking`** Observe how the form behaves when it is resized. Try altering the directions which the gadgets are docked.

```
setup form !!exampleDocking dialog resizeable
  !this.formTitle = |Dock and Anchor|
  !buttpos = |xmax.f1 - size ymax.f1 - size|
  frame .f1 anchor ALL width 30 height 5
    button .but1 |Dock TOP| dock TOP
    button .but2 |Anchor B+R| at $!buttpos anchor R+B
  exit
exit
```

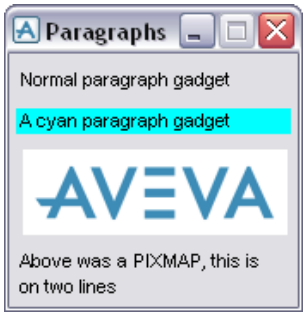


2.6 Paragraph Gadgets

Paragraph gadgets are simple named gadgets which allow a piece of **TEXT** or a **PICTURE** (PIXMAP) to be displayed on a form. It is a passive gadget that cannot be selected by the user so has no callback. Paragraph gadgets are typically used display information or images.

Show the example by typing **show !!exampleParagraphs**. Notice how the value of the last gadget was set during the **CONSTRUCTOR** method, it was only given a size to reserve the space during definition.

```
setup form !!exampleParagraphs
  !this.formTitle  = |Paragraphs|
  para .para1 text |Normal paragraph gadget| width 20
  para .para2 at x0 ymax.para1 backg 6 text |A cyan paragraph gadget| width 20
  para .para3 at x0 ymax.para2 pixmap width 154 height 50
  para .para4 at x0 ymax.para3 + 0.5 text || wid 20 hei 2
exit

define method .exampleParagraphs()
  -- Update the displayed text using CONSTRUCTOR method
  !this.para4.val = |Above was a PIXMAP, this is on two lines|
  -- Use built-in method to apply picture
  !this.para3.AddPixmap(|c:\temp\pmllib\icons\aveva.png|)
endmethod
```



Refer to the Reference Manual and Guide for more information about paragraph gadgets

2.7 Button Gadgets

Button gadgets are typically used to invoke an action or to display a child form. Its **CALLBACK** can call a **LOCAL METHOD**, **GLOBAL FUNCTION** or **OBJECT METHOD**. If a callback and a child form are both specified, the callback command will be run before the child form is displayed.

Show the example by typing **show !!exampleButtons**. A linklabel style button is now available and has the same appearance as a hyperlink. Notice how the value of the toggle button changes.

```
setup form !!exampleButtons
  !this.formTitle = |Buttons|
  button .butt1 |Normal|
  button .butt2 linklabel |LinkLabel| wid 6
  button .butt3 |Green| backg 5
  button .butt4 |Deactive|
  button .butt5 |form| form !!exampleParagraphs
  button .butt6 toggle backg 4 call |!this.check()| pixmap wid 31 hei 21
exit

define method .exampleButtons()
  -- Deactivate butt3 in CONSTRUCTOR method
  !this.butt4.active = FALSE
  -- Use built in method to apply pictures
  !this.butt6.AddPixmap(|c:\temp\pmllib\icons\off.png|, |c:\temp\pmllib\icons\on.png|)
endmethod

define method .check()
  -- Return the toggle button value to the command line
  !check = !this.butt6.val
  $p $!check
endmethod
```

Refer to the Reference Manual and Guide for more information about button gadgets

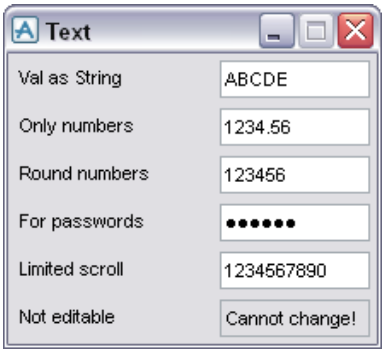
2.8 Text Entry Gadgets

A text input gadget provides the user a way of entering a single value into PDMS. A TEXT gadget needs two aspects to be defined:

- **WIDTH** – determines the displayed number of characters. An optional scroll width can also be specified
- **TYPE** – determines the type of the variable created when inputting a value. This is important when PML uses the variable. You may also supply a **FORMAT** object (explained below) to format the value entered (e.g. 2 d.p number)

Show the example by typing **`show !!exampleTexts`**. Test the various text gadgets by entering values. The benefit of making a text gadget uneditable (rather than deactivated) is so the user can select its contents. A de-active text box will be full greyed out and unselectable.

```
setup form !!exampleTexts
  !this.formTitle = |Text|
  path down
  text .txt1 |Val as String | width 10 is STRING
  text .txt2 |Only numbers  | width 10 is REAL format !!REALFMT
  text .txt3 |Round numbers | width 10 is REAL format !!INTEGERFMT
  text .txt4 |For passwords | width 10 NOECHO is STRING
  text .txt5 |Limited scroll| width 10 scroll 1 is STRING
  text .txt6 |Not editable  | width 10 is STRING
exit

define method .exampleTexts()
  !this.txt6.val = |Cannot change!|
  -- Make txt6 uneditable
  !this.txt6.setEditable(FALSE)
endmethod
```



Refer to the Reference Manual and Guide for more information about text gadgets

2.9 Format Object

A **FORMAT** object manages the information needed to convert a number (always in mm) to a STRING. It can also be used apply a format to a text gadget. A format objects are usually defined as global variables so that they are available across PDMS. For example, type the following onto the command line:

```
!!oneDP = object FORMAT()
!!oneDP.dp = 1
q var !!oneDp
```

There are four standard **FORMAT** objects which are already defined in standard PDMS:

!!distanceFmt	For distance units
!!boreFmt	For Bore Units
!!realFmt	To give a consistent level of decimal places on real numbers
!!integerFmt	To force real numbers to be integers(0 dp Rounded)

To find out more information about these **FORMAT** object, query them as global variables on the command line **`q var !!boreFmt`**

For example the number of decimal places displayed using !!RealFmt could be set **`!!realFmt.dp = 6`** the default value is 2.

These standard format objects are used within the forms of PDMS. Changing the definition of these objects will change the way standard product behaves.

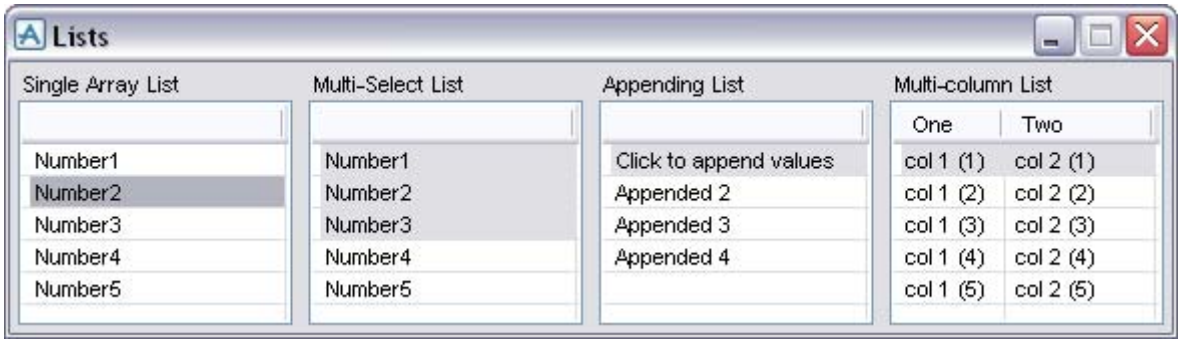
2.10 List Gadgets

A **LIST** gadget presents and **ARRAY** of values to the user. This can be a **SINGLE** or **MULTI-DIMENSIONAL ARRAY**

All the values in the gadget are set by assigning an **ARRAY**. **ARRAY** variables can be applied to a **LIST** at any time. The choice between MULTIPLE, **COLUMN** and **SINGLE** has to be made when the gadget is defined. The values within a **COLUMN** list (headings and values) are set by using built-in gadget methods.

Show the example by typing **`show !!exampleLists`**. Test the different lists by trying to select rows.

```
setup form !!exampleLists
  !this.formTitle = |Lists|
  !vshap = |width 15 height 6|
  list .lst1 |Single Array List| call |!this.value()| SINGLE ZEROSEL $!vshap
  list .lst2 |Multi-Select List| MULTI $!vshap
  list .lst3 |Appending List| call |!this.append()| $!vshap
  list .lst4 |Multi-column List| $!vshap
exit

define method .exampleLists()
  do !I from 1 to 5
    !values[!I] = |Number| & !I
    !rtext[!I] =  !I.string()
    do !J from 1 to 2
      !multi[!I][!J] = |col | & !J & | (| & !I & |)|
    enddo
  enddo
  !this.lst1.dtext = !values
  !this.lst1.rtext = !rtext
  !this.lst2.dtext = !values
  !single[1] = |Click to append values|
  !this.lst3.dtext = !single
  !this.lst4.setRows(!multi)
  !heading = |One Two|
  !this.lst4.setHeadings(!heading.split())
endmethod

define method .value()
  !dtext = !this.lst1.selection('Dtext')
  !rtext = !this.lst1.selection('Rtext')
  $p Selected Dtext = $!<dtext> Rtext = $!<rtext> (hidden!)
endmethod

define method .append()
  !nextLine = !this.lst3.dtext.size() + 1
  !val = |Appended | & !nextLine
  !this.lst3.add(!val)
endmethod
```



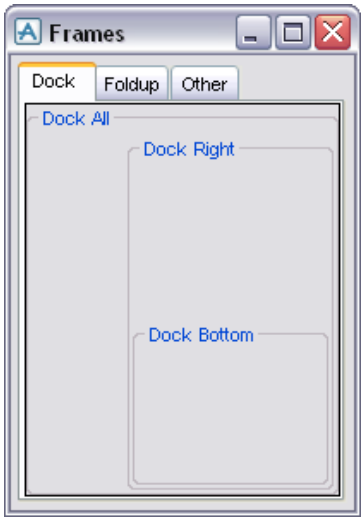
Refer to the Reference Manual and Guide for more information about list gadgets.

2.11 Frame Gadgets

A **FRAME** is a cosmetic gadget which is used to surround a group of similar gadgets. This helps with organisation, positioning and user experience. A frame can also be declared as a **FOLDUP**, **PANEL**, **TABSET** or even a **TOOLBAR** (when defined for the main PDMS window).

Show the example by typing **show !!exampleFrames**. The example shows the various types of frames and the members/methods available. Notice how the frames immediately below the **TABSET** are its tabs.

```
setup form !!exampleFrames dialog resizeable
  !this.formTitle = |Frames|
  frame .tabset TABSET anchor ALL wid 15 hei 5
    frame .f1 |Dock| at 0 0 dock FILL
      frame .fA |Dock All| dock FILL
        frame .fB |Dock Right| dock RIGHT wid 10
          frame .fC |Dock Bottom| dock B hei 4
          exit
        exit
      exit
    exit
    frame .f2 |Foldup| at 0 0 dock FILL
      button .but1 linklabel |Show Panel Frame...| at 0 0 call |!this.showD()|
      frame .fD panel |Panel Frame| at x0 ymax.but1 dock FILL
        frame .fE foldup |Foldup 1| at x1 ymin.fD + 1 wid 20 hei 3
          button .but2 linklabel |Foldup Panel| call |!this.fold()|
        exit
        frame .fF foldup |Foldup 2| at xmin.fE ymax.fE wid 20 hei 3
          para .para1 text |Inside a frame|
        exit
      exit
    exit
    frame .f3 |Other| at 0 0 dock FILL
      !pos = |at xcen.fG - 0.5 * size ymax|
      frame .fG |Deactive| wid 21 hei 4
        button .but3 |Try and click!| at x 2 y 1
      exit
      button .but4 toggle |Show Frame| $!pos backg 8 call |!this.showF()|
      frame .fH |Hidden Frame| at xmin.fG ymax.but4 width.fG hei.fG
      exit
    exit
  exit
exit

define method .exampleFrames()
  !this.fD.visible = FALSE
  !this.fE.expanded = FALSE
  !this.fG.active = FALSE
  !this.fH.visible = FALSE
endmethod

define method .fold()
  !this.fE.expanded = FALSE
endmethod

define method .showD()
  if !this.but1.val.eq(TRUE) then
    !this.fD.visible = TRUE
    !this.but1.tag = |Hide Panel Frame...|
  else
    !this.fD.visible = FALSE
    !this.but1.tag = |Show Panel Frame...|
  endif
endmethod
```



(continued onto next page)

```
define method .showF()
  if !this.but4.val.eq(TRUE) then
    !this.fH.visible = TRUE
    !this.but4.tag = |Hide Frame|
  else
    !this.fH.visible = FALSE
    !this.but4.tag = |Show Frame|
  endif
endmethod
```

ⓘ *When creating a FRAME gadget, for every **FRAME** there must be an associated **EXIT**.*

If insufficient exits are provided, this will cause an error and the form **WILL NOT LOAD**. As the error occurred inside the **FORM DEFINITION**, the command line will still be in form definition mode and will not function as usual. To exit this mode, type **EXIT** on the command line until and ERROR is received. This will mean that form definition mode has been exited and normal commands will work again.

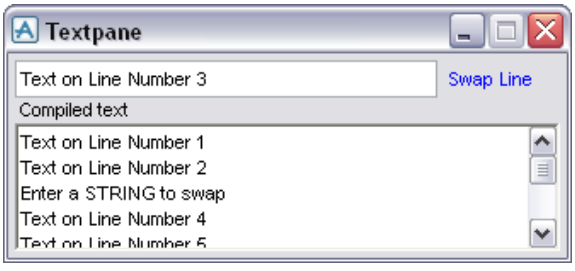
It is good practise to provide 2 spaces when working inside a code block. This provides an easy way to spot missing exits.

2.12 Textpane Gadgets

A **TEXTPANE** gadget provides an area on a form into which a user may edit multiple lines of text and cut/paste from elsewhere on the PML screen. The inputted value is stored as an **ARRAY of STRINGS** and can be set and read from the gadget.

Show the example by typing **`show !!exampleTextpane`**. Choose different lines inside the textpane and press the button. The method reads the cursor position inside the gadget and it will swap that line. The textpane cannot be given a callback so any interaction has to be through something else.

```
setup form !!exampleTextpane
  !this.formTitle = |Textpane|
  !this.initCall  = |!this.init()|
  text .txt1 width 30 is STRING
  button .but1 linklabel |Swap Line| at xmax.txt1 + 0.5 y 0 call |!this.swap()| wid 7
  textpane .txtp |Compiled text| at x 0 ymax wid 40 hei 5
exit

define method .init()
  !this.txt1.val = |Enter a STRING to swap|
  do !n from 1 to 10
    !val[!n] = |Text on Line Number | & !n
  enddo
  !this.txtp.val = !val
endmethod

define method .swap()
  !lineNO = !this.txtp.curPos()
  !line = !this.txtp.line(!lineNo[1])
  !this.txtp.setLine(!lineNo[1], !this.txt1.val)
  !this.txt1.val = !line
endmethod
```



2.13 Option Gadgets

An **OPTION** gadget offers a single choice from a list of items. It may contain either **PIXMAPS** or **STRINGS**, but not a mixture. The gadget displays the **CURRENT** choice in the list. When the user presses the option gadget, the entire set of items is shown as a drop-down list and the user can then select a new item by dragging cursor to the option required. There is now also a COMBO gadget that allows the user to type in a value. If used in conjunction with an open callback (explained in Chapter XXX), it can select from the list for the user. This would be very useful when there are a long number of options.

The width of a text option gadget must be specified. A tag name is optional and is displayed to the left of the gadget. The available options is stored as an **ARRAY of STRINGs** as the gadgets **DTEXT** or **RTEXT** and can be updated by altering this array.

Show the example by typing **`show !!exampleOptions`**. Test the COMBO gadget by typing in part of the required colour and press enter.

```
setup form !!exampleOptions
  !this.formTitle = |Options|
  path right
  option .opt1 |Normal| width 7
  combo  .opt2 |Combo | width 7
  option .opt3 |Pixmap| pixmap width 16 height 16
exit

define method .exampleOptions()
  !dtext = |Red Yellow Green Cyan Blue Purple Pink|
  !this.opt1.dtext = !dtext.split()
  !this.opt2.dtext = !dtext.split()
  do !n index !this.opt1.dtext
    !files[!n] = |/c:\temp\pmllib\icons\| & !n & |.gif|
  enddo
  !this.opt3.dtext = !files
  !this.opt3.rtext = !dtext.split()
  !this.opt2.callback = |!this.setColour(|
endmethod

define method .setColour(!gad is gadget, !event is STRING)
  if !event.eq('VALIDATE') then
    !userInput = !this.opt2.displayText()
    do !n index !this.opt2.dtext
      !chrs = !userInput.length()
      !test = !this.opt2.dtext[!n].upcase().substring(1, !chrs)
      if !userInput.upcase().eq(!test) then
        !this.opt2.val = !n
        break
      endif
    enddo
  endif
endmethod
```

Try querying the Rtext of the pixmap gadget by using a built-in gadget method:

```
q var !!exampleOptions.opt3.selection()
```

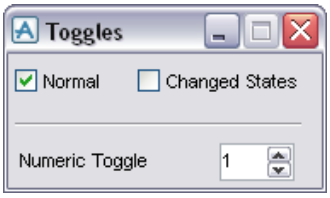
2.14 Toggle Gadgets

TOGGLE gadgets are used for independent on/off settings. This means they are used in situations where the user has two choices i.e. is it bold or not? Is it on or off? A toggle will return a BOOLEAN state (stored under its val member), but can also store different descriptions for selected and unselected.

NUMERIC INPUT gadgets can be used to allow users to enter real values within a range. The user can also use the supplied toggles to alter the value by the defined step.

Show the example by typing **show !!exampleToggles**. Click the toggles and observe the values on the command line. The method is given an argument of the gadget which called it. This means that the correct value is returned. Notice how a **LINE** gadget has been used to divide the form.

```
setup form !!exampleToggles
  !this.formTitle = |Toggles|
  path right
  toggle .tog1 tagwid 5 |Normal| call |!this.state(!this.tog1)|
  toggle .tog2 tagwid 10 |Changed States| call |!this.state(!this.tog2)| states |N| |Y|
  line   .line at xmin.tog1 ymax.tog1 horiz wid 21 hei 1
  numeric .num |Numeric Toggle| at xmin.tog1 ymax.line range 0 10 step 1 NDP 0 wid 3
exit

define method .state(!gad is GADGET)
  if (!gad.val) then
    q var !gad.onvalue
  else
    q var !gad.offvalue
  endif
endmethod
```



2.15 Radio Gadgets

A **RADIO GROUP** is used to give a user a single choice between a small fixed number of choices. Two objects can be used to define a radio group: **RGROUP** or **RTOGGLE**. An **RGROUP** element defines the entire object and the tags contained; an **RTOGGLE** is contained within a normal **FRAME** object. **RGROUP** objects can be displayed vertically or horizontally. **RTOGGLE** objects can be arranged within a FRAME as required and can be placed alongside other gadgets

An **RGROUP** object has been deprecated and maybe withdrawn in the future. Use an **ROGGLE** in preference for new and upgrading code.

Show the example by typing **show !!exampleRGroups**. Notice how similar the gadgets appear and behave. Comment out the **CONSTRUCTOR** method to see that RTOGGLE elements can be given different callbacks, instead of the callback on the containing frame.

```
.
setup form !!exampleRGroups
  !this.formTitle = |Radio Groups|
  rgroup .vert |RGroup| FRAME vertical callback |!this.rg(!this.vert)|
    add tag |Left| select |L|
    add tag |Centre| select |C|
    add tag |Right| select |R|
  exit
  frame .rtog |RToggle| at xmax + 1 y 0
    path down
    rToggle .left |Left| states || |L|
    rToggle .cen |Centre| call |q var !this.cen.onvalue| at ymax - 0.1 states || |C|
    rToggle .righ |Right| at ymax - 0.1 states || |R|
  exit
exit

define method .exampleRGroups()
  !this.rtog.callback = |!this.rt(!this.rtog)|
endmethod

define method .rg(!gad is GADGET)
  q var !gad.selection()
endmethod

define method .rt(!gad is GADGET)
  !rtog = !gad.rtoggle(!gad.val)
  q var !rtog.onvalue
endmethod
```

2.16 User-Defined Form Members

As a **FORM** is an **OBJECT**, it may be given **MEMBERS**. This is a useful way to store data, effectively creating global variables. The benefit of this method is that data is global without having large numbers of individual variables.

Form members replaces the previous method of **USERDATA** (PML 1 style data storage), and are given an object-type. These variables have the same lifetime as the form and are deleted when the form itself is unloaded.

Show the example by typing **`show !!exampleFormMembers`**. It shows how a form can be assigned an element on showing the form and how this element can be updated.). Every time the update button is pressed, the element is stored in the member **.storage**.

```
setup form !!exampleFormMembers resize
  !this.formTitle = |Form Members|
  !this.initcall  = |!this.init()|
  para .ceName wid 30 hei 1
  list .attrib |Attributes:| at x 0 ymax anchor all wid 35 hei 20
  button .update linklabel |Update| at xmax.ceName ymin.ceName wid 4
  member .ceRef   is DBREF
  member .storage is ARRAY
exit

define method .exampleFormMembers()
  !this.update.callback = |!this.init()|
  !headings = |Attribute Value|
  !this.attrib.setHeadings(!headings.split())
endmethod

define method .init()
  !this.ceRef = !!CE
  !this.ceName.val = !!CE.FLNN
  !this.storage.append(!this.ceRef)
  !this.fill()
endmethod

define method .fill()
  !atts = !this.ceRef.attributes()
  do !n index !atts
    !dtext[1][!n] = !atts[!n].string()
    !dtext[2][!n] = !this.ceRef.attribute(!atts[!n]).string()
  enddo
  !this.attrib.setcolumns(!dtext)
endmethod
```

Investigate the information stored on the form members by typing:

```
q var !!exampleFormMembers.ceRef
q var !!exampleFormMembers.storage
```

2.17 Tooltips

Tooltips are small help boxes which pop up when you move the mouse over an active gadget. Tooltips are typically used to provide more information to the user (for example, on a pixmap button). An example of the syntax is as follows:

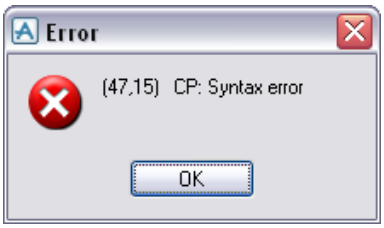
```
BUTTON .B1 |SAVE| PIXMAP TOOLTIP |Save Work|
```

Refer to the specific gadgets in the PDMS Software Customisation Reference Manual

Exercise 1 - Working with a form

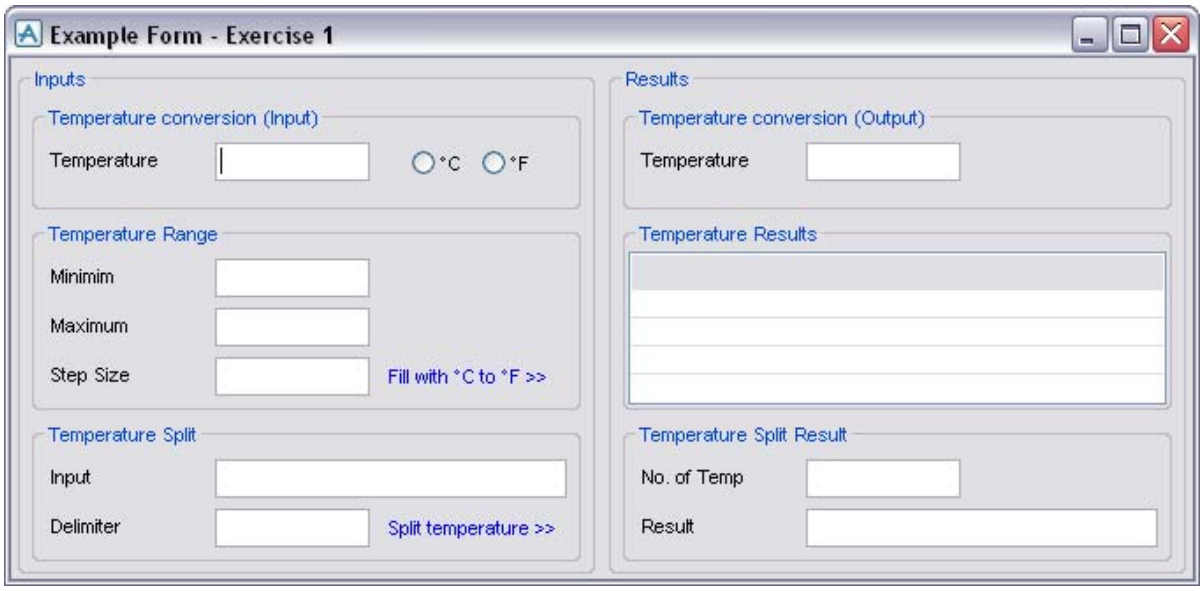
1

- You have been provided with a form called !!c2ex1. To show the form, type **`show !!c2ex1`**
- The form has not been maintained and there are a number of bugs in the form. At least 3 are preventing it from being shown and at least 2 are causing the wrong information to be displayed.
- Debug the form so that it can be shown and displays the correct information. The fixes are listed in Appendix B



2

- The form demonstrates poor gadget arrangement and impacts on the user experience. Work through the gadgets and improve their arrangement and sizes.
- An example of the aligned form is shown in the below screenshot.

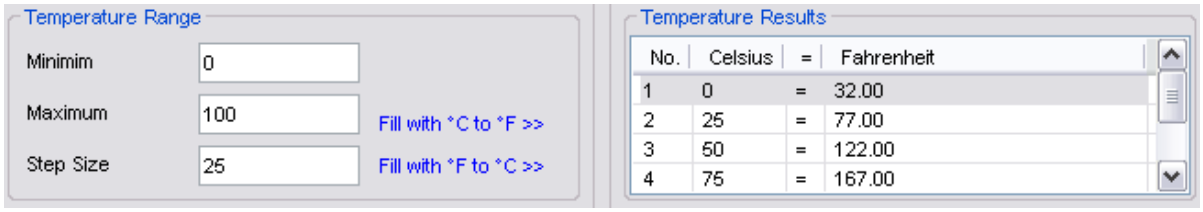


3

- Write a method that will run when the form is shown. This method shall be used to fill default values into the input frame.
- When the input values are set, the method should then run the methods that fill in the output frame.

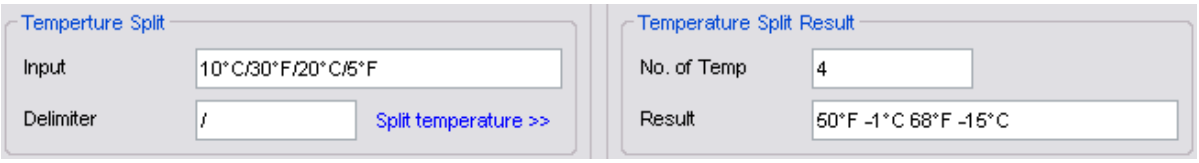
4

- Add a new button to the form that will fill the temperature conversion chart with Fahrenheit to Celsius values
- The method has already been written. Consider which method needs to be called and what argument needs to be passed



5

- There is currently no method available to split the temperature string the in lower frames.
- Write a method to do the following:
  - Split the input string based on the delimiter
  - Read the individual temperatures and convert them (this will depend on °C or °F)
  - Compile the converted values as a space separated string
  - Write the compiled string back to the form and display how many temperatures
- This method should be run when the form is shown or the button is pressed.



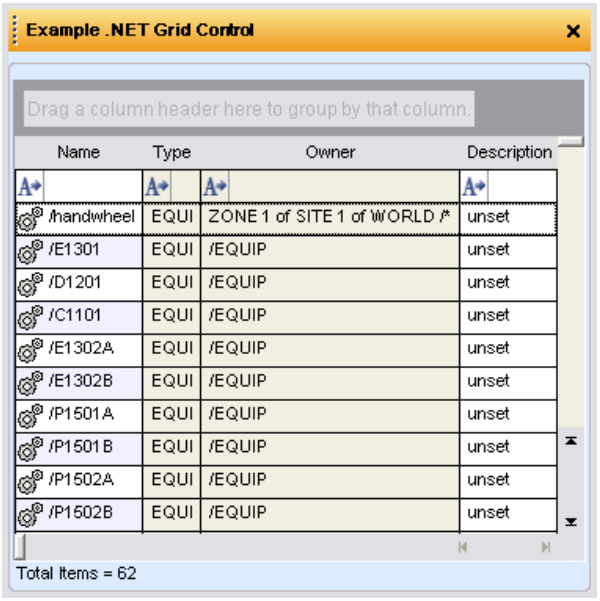
An example of the completed form can be found in Appendix B

2.18 PML.NET Grid Control Gadget

Using AVEVA Plant 12, it is now possible to customise the product using .NET through .NET controls and objects. PML.NET allows you to include .NET objects and invoke their methods using PML. It is possible to add container objects to a conventional PML form creating a hybrid of PML & .NET. The container is treated as a normal form gadget (i.e syntax graph) but can be filled with a .NET object

A common example of an available .NET object is the **Grid Control** gadget. Despite appearing similar to a **LIST** gadget, the Grid Control gadget has a large number of methods available to it allowing it to behave more like a spreadsheet. Some of the advantages are as follows:

- Data in the grid can be selected, sorted and filtered.
- Data in the grid can be exported to/imported from a Microsoft Excel file
- Grid contents can be previewed and printed directly
- The colour of rows, columns or cells can be set (including icons)
- The values of entire rows/columns can be extracted/set
- The contents of the grid can be filled based on DB elements and their attributes



Show the example by typing **`show !!exampleGridControl`**.

Before a .NET control can be used in a form, it first has to be imported into PDMS using the IMPORT syntax. The required **.dll** file has to be loaded and the namespace specified. For example:

```
import 'GridControl'
using namespace 'Aveva.Pdms.Presentation'
!netObject = object NETGRIDCONTROL()
```

Data is applied to a **Grid Control** gadget through the use of a **NETDATASOURCE** object. While defining this object, the required data is collected and formatted for the grid. The data is applied to the grid by using its **.BindToDataSource()** method. For example:

```
var !equip collect all EQUI
!attribs = |NAME TYPE OWNER AREA|
!data = object NETDATASOURCE('Example Grid, !attribs.split(), !equip)
!this.exampleGrid.bindToDataSource(!data)
```

In this example, the NetDataSource object is collecting information from the database. There are other methods available as listed in Appendix A. For further information the .NET Cutomisaton User Guide, available within the 12.0 manual

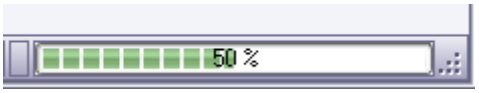
2.19 Progress Bars and Interrupting Methods

The standard PDMS global object **!!FMSYS** (Forms & Menus System) provides two pieces of functionality which can be applied when designing a form:

- An progress bar (appearing at the bottom right of the main program)
- The ability to interrupt a method

The progress bar is activated by passing a real number to the **.setProgress()** method. For example:

```
!!FMSYS.setProgress(50)
```



Passing it a zero argument will cause the progress bar to disappear

To identify a gadget which can be used to interrupt a method, it must be passed as an argument to the **.setInterrupt().** The method can then check the !!FMSYS object to see if the gadget has been pressed through its **.interrupt()** method

Show the example by typing **show !!exampleProgressBar**. Press the start button and press it again to stop the method early

```
setup form !!exampleProgressBar dialog NoAlign
  button .b1 at xmin ymax call |!this.counter()| pixmap wid 100 hei 100
  numeric .num at xmax - size ymax + 0.1 range 0 10000 step 1 ndp 0 val 0 wid 4
exit

define method .exampleProgressBar()
  !this.b1.addPixmap(!!pml.getpathname('gobutton.png'))
endmethod

define method .counter()
  !!FMSYS.setInterrupt(!!exampleProgressBar.b1)
  !this.b1.addPixmap(!!pml.getPathName('stopbutton.png'))
  !this.b1.refresh()
  do !n from 1 to 10000
    !this.num.val = !n
    !this.num.refresh()
    !percent = !n / 100
    -- Check if the method has been interrupted.  Break if it has
    if (!!FMSYS.interrupt()) then
      !!alert.message(!n & | loops were completed - | & !percent.string(|D1|) & |%|)
      break
    endif
    !!FMSYS.setProgress(!percent)
  enddo
  !this.b1.addPixmap(!!pml.getPathName('gobutton.png'))
  !!FMSYS.setProgress(0)
endmethod
```

2.20 Open Callbacks

An **OPEN CALLBACK** is a way of providing change information about the gadget the user is interacting with. It means that for every interaction event, the callback will be called. Two pieces of information are passed during an open callback:

- the gadget being interacted
- a keyword (as a STRING).

These keywords indicate the event which caused the callback. Different keywords will be generated by different gadgets under different circumstances e.g. a multi-selection gadget may have a 'SELECT' 'UNSELECT', as well as the more standard 'START' 'STOP'

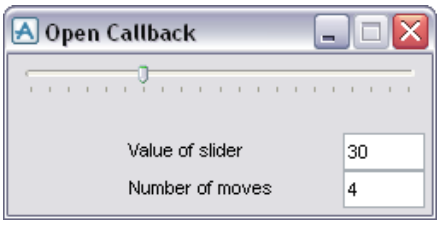
PDMS will recognise an open callback if a method only has one bracket e.g. **`call |!this.opencall(|`**

For an open callback to work, there must be an appropriate method defined. An open callback method must be able to accept two arguments (1) the gadget and (2) the keyword

e.g. **`define method .opencall(!a is GADGET, !b is STRING)`**

As the method is called at every significant event, the open callback method should be written to recognise keywords. The example has been written around the slider gadget. Show the example by typing **`show !!exampleOpenCallback`**. Investigate the form by sliding the slider. Review the methods to identify how it works. Also, revisit the !!exampleOptions to review that opencallback.

```
setup form !!exampleOpenCallback
  !this.formTitle = |Open Callback|
  slider .slide anchor L+R horizontal range 0 100 step 5 val 50 width 30
  text .text |Value of slider| at xmax - size ymax + 1 anchor B+R width 5 is REAL
  text .moves |Number of moves| at xmax.text - size ymax anchor B+L width 5 is REAL
  member .store is REAL
exit

define method .exampleOpenCallback()
  !this.slide.callback = |!this.slideMove(|
  !this.moves.val = 0
  !this.store = 0
  !this.text.val = !this.slide.val
endmethod

define method .slideMove(!gad is GADGET, !val is STRING)
    !this.text.val = !gad.val
    !this.text.refresh()
    if !val eq |MOVE| then
      !this.store = !this.store + 1
    elseif !val eq |STOP| then
      !this.moves.val = !this.store
      !this.store = 0
    endif
endmethod
```



The **.refresh()** method has been applied to the .text gadget to ensure its value updates as the slider is moved. The number of moves is stored as a member of the form. This saves a new global variable from being defined and means the increasing value can be shared between the open callbacks.

ⓘ *An open callback can be given to any gadget that accepts a callback. Different gadgets may generate different event keywords*

2.21 Menus

Menu objects are used to provide options to a user and can be applied to a form as either a:

- Menu bar across the top of the form
- Popup menu on a gadget

In both cases, a menu object must be defined to represent the options part of the menu bar or popup menu.

2.21.1 Defining a Menu Object on a Form

Within the form definition, the form method .newMenu() creates a named menu object. Once the object is defined, the.add() method on the menu object can be used to add named menu fields. A menu field can do one of three things:

- Execute a callback
- Display a form
- Display a sub-menu

You can also add a visual separator between fields.

```
!menu = !this.newmenu(|exampleMenu|, |MAIN|)
!menu1.add(|CALLBACK|,|Query|,|q atts|)
!menu1.add(|FORM|,|Progress Bar…|, |exampleProgressBar|)
!menu1.add(|SEPARATOR|)
!menu1.add(|MENU|,|Pull-right1|,|Pull1|)
```

This creates a menu object called exampleMenu with 3 fields (Query, Hello… and Pull-right1) and a separator between the last two fields

- The Query field when picked will execute the **CALLBACK** command ‘q att’ (prints the CE attributes to the command window
- The Hello... field when picked will load and display the **FORM** from the previous section !!exampleProgressBar. By convention, the text on a menu field leading to a form ends with three dots, which you must include with the text displayed for the field.
- The **SEPARATOR**, usually a line, will appear after the previous field.
- The Pull-right1 **MENU** field when picked will display the sub-menu !this.Pull1 to its right. A menu field leading to a sub-menu ends with a **>** symbol (this is added automatically)

2.21.2 Defining a Bar Menu on a Form

Forms may have a bar menu gadget which appears as a row of options across the top of the form. A bar menu is defined within form definition and specifies the options the user has to choose from. There are three types that can be added:

- Choose – displays a user-defined menu object (reference by its name)
- Window – displays a list of the open forms
- Help – displays the standard help options

After the bar command, use the bar object method .add() to add extra options to the bar menu. As there can only be one bar menu, !this.bar refers to the bar menu. For example:

```
bar
  !this.bar.add(|Choose|,|exampleMenu|)
  !this.bar.add(|Window|,|Window|)
  !this.bar.add(|Help|,|Help|)
```

ⓘ *Bar menus can only be added to forms which are not dockable*

2.21.3 Defining a Popup Menu on a Form

Pop-up Menu is a MENU object that are displayed from a gadget by right-clicking on it. This is a useful method of providing the user with relevant functionality relating to the associated gadget.

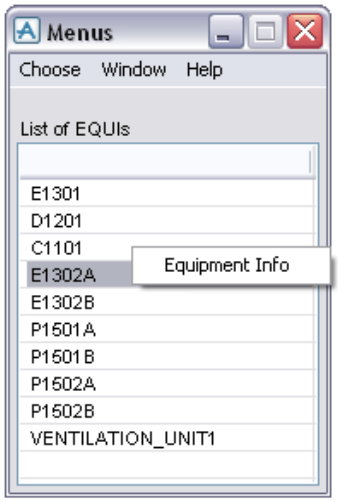
```
!this.exampleList.setPopup(!this.exampleMenu)
```

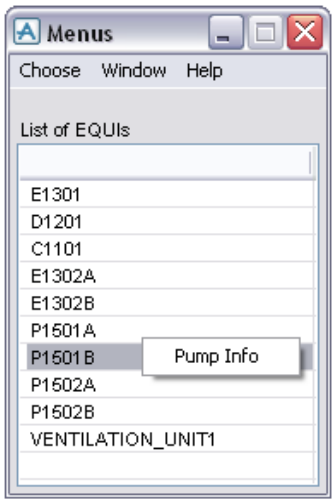
The MENU object is applied to a gadget through the .setPopup() method (available on a number of gadgets). The MENU object can be applied at any time, so popup menus can change to ensure the content is always relevant.

2.21.4 Adding Menus Objects to a Form

Show the example by typing show !!exampleMenus.

```
setup form !!exampleMenus
  !this.formTitle = |Menus|
  !this.initcall  = |!this.init()|
  bar
  !this.bar.add(|Choose|,|exampleBarMenu|)
  !this.bar.add(|Window|,|WINDOW|)
  !this.bar.add(|Help|,|HELP|)

  list .list |List of EQUIs| callback |!this.pickList()| wid 20 hei 12

  !menu = !this.newMenu(|exampleBarMenu|)
  !menu.add(|CALLBACK|,|Query|,|q atts|)
  !menu.add(|FORM|,|Progress Bar...|, |exampleProgressBar|)
  !menu.add(|SEPARATOR|)
  !menu.add(|MENU|,|Pull-right1|,|Pull1|)

  !menu = !this.newMenu(|exampleEquiMenu|)
  !menu.add(|CALLBACK|, |Equipment Info|, |$p EQUI command|)

  !menu = !this.newMenu(|examplePumpMenu|)
  !menu.add(|CALLBACK|, |Pump Info|, |$p PIPE command|)
exit

define method .init()
  -- Collect the equipment from the Stabilizer Plant
  var !equipment coll all EQUI for /EQUIP
  var !equipmentNames eval FLNN for all from !equipment
  !this.list.dtext = !equipmentNames
  !this.list.rtext = !equipment
endmethod

define method .pickList()
  -- Set the popup menu based on the the first letter of name
  if !this.list.selection(|DTEXT|).substring(1,1).eq(|P|) then
   !this.list.setPopup(!this.examplePumpMenu)
  else
    !this.list.setPopup(!this.exampleEquiMenu)
  endif
endmethod
```



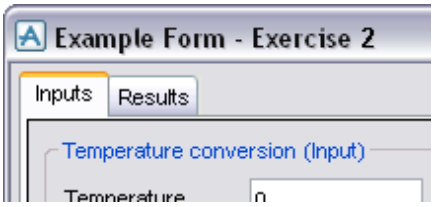


Refer to the PDMS Customisation Reference Manual for the details of which gadgets permit a popup menu.

Exercise 2 – Extending the form

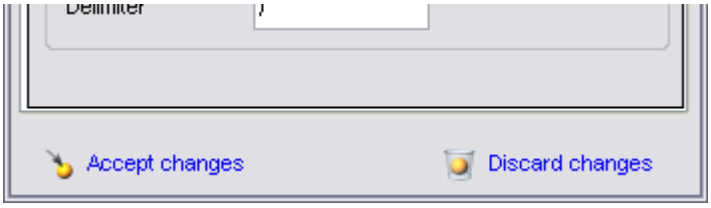
1

- Save the form from the previous exercise as c2ex2.pmlfrm and update the name of the form and name of the constructor method. Type **PML REHASH ALL** so PDMS finds the new file
- Upgrade the frames from Exercise 1 to a tabset
- Remove the callbacks from the existing gadgets and any buttons from the form. The methods should now be called through an **OPEN CALLBACK** on the Results tab. When the tab is shown, the methods run.



2

- Add an **ARRAY** member to the form that will store the values of the input gadgets. This will provide the ability to reset the values if they are changed.
- Write a method that will allow data to be applied to and taken from this member.
- Add two buttons to the base of the form. One to update the stored values (Accept changes) and the other will apply the stored values to the gadgets (Discard changes)

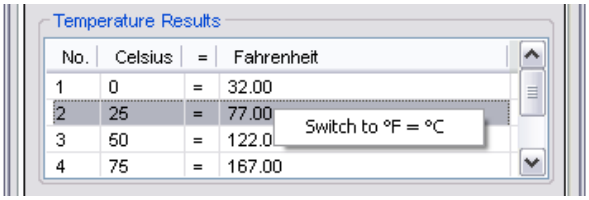


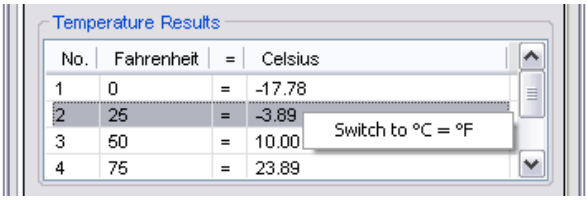
- The buttons will be **linklabels** and should call the method to store/reset data. To the left of the buttons there should be **16x16 pixmap paragraph**. Using the **.addPixmap()** method, apply standard PDMS images to them.

```
.addPixmap(!!pml.getPathName('accept.png'))
.addPixmap(!!pml.getPathName('discard.png'))
```

3

- As the buttons have been removed from the form, only one type of conversion can be applied to the List gadget (depending on which method the open callback calls)
- Add a **POPUP MENU** to the list gadget which will allow the other type of conversion to be done





- There should be two new menu objects created on the form; one will run the method as Celsius and the other as Fahrenheit. After a choice has been made, the popup menu needs to be swapped to the other one. Consider what methods will need to be called and how the popup menus can be managed.

An example of the completed form can be found in Appendix B

3 PML Objects

3.1 Built in PML Object Types

There are a large number of standard objects which are supplied with PDMS. These include All Variable types, BORE, DIRECTION, DBREF, FORMAT, MDB, ORIENTATION, POSITION, FILE, PROJECT, SESSION, TEAM, USER, ALERT, FORM, all form Gadgets and various graphical aid objects.

Each object has a set of built in methods for setting or formatting the object contents. The following are the objects covered by the Software Customisation Reference Manual:

ARRAY	DATEFORMAT	FORMAT	REAL
BANNER	DATETIME	LOCATION	REPORT
BLOCK	DB	MACRO	SESSION
BOOLEAN	DBREF	MDB	STRING
BORE	DBSESS	OBJECT	TABLE
COLLECTION	DIRECTION	ORIENTATION	TEAM
COLUMN	EXPRESSION	POSITION	UNDOABLE
COLUMNFORMAT	FILE	PROJECT	USER

The following objects from the reference manual cover forms and menus:

ALERT	FORM	OPTION	TEXTPANE
BAR	FRAME	PARAGRAPH	TOGGLE
BUTTON	LINE	RTOGGLE	VIEW ALPHA
COMBOBOX	LIST	SELECTOR	VIEW AREA
CONTAINER	MENU	SLIDER	VIEW PLOT
FMSYS	NUMERIC	TEXT	VIEW VOLUME

The following objects from the reference manual cover 3D geometry:

ARC	LOCATION	POINTVECTOR	PROFILE
LINE	PLANE	POSTEVENTS	RADIAL GRID
LINEARGRID	PLANTGRID	POSTUNDO	XYPOSITION

3.2 Methods Available to All PML Objects

In addition to the object specific methods, there are methods that are common to all objects. The following table lists the methods available to all objects:

Name	Result	Purpose
Attribute('Name')	ANY	To set or get a member of an object, providing the member name as a STRING.
Attributes()	ARRAY OF STRINGS	To get a list of the names of the members of an object as an array of STRING.
Delete()	NO RESULT	Destroy the object - make it undefined
EQ(any)	BOOLEAN	Type-dependent comparison
LT(any)	BOOLEAN	Type-dependent comparison (converting first to STRING if all else fails)
Max(any)	ANY	Return maximum of object and second object

Min(any)	ANY	Return minimum of object and second object
NEQ(any)	BOOLEAN	TRUE if objects do not have the same value(s)
ObjectType()	STRING	Return the type of the object as a string
Set()	BOOLEAN	TRUE if the object has been given a value(s)
String()	STRING	Convert the object to a STRING
Unset()	BOOLEAN	TRUE if the object does not have a value

Refer to the Software Customisation Reference Manual for further details.

3.3 The FILE Object

The **FILE** object is an example of how older functionality has been replaced with a PML 2 style object. This object replaces the 'openfile' 'readfile' 'writefile' 'closefile' syntax and provides increased functionality (file path, if the file is open etc). It is now possible to read or write to a file in a single operation.

To create a file object:

```
!input = object file('C:\FileName')
!output = object file('C:\FileName.out')
```

To Open a file:

```
!output.open('WRITE')
```
Avaiable options = READ, WRITE, OVERWRITE, APPEND

3.3.1 Using FILE Objects

To read a line from file:

```
!line = !input.readRecord()
```
File must be open

To write a line to file:

```
!output.writeRecord(!line)
```
File must be open

To read all the input file:

```
!fileArray = !input.readFile()
```
Files are opened and Closed automatically

To write all of the data to file

```
!output.writeFile('WRITE',!fileArray)
```
Files are opened and Closed automatically

The .ReadFile() method has a default maximum file size which it can read. This can be increased by passing the method a REAL argument (representing the number of lines in the file)

3.3.2 Opening a FILE Object in Notepad

You could use the **SYSCOM** command to open an external program like Notepad, by adding an & to the end of the line will opens a new process. If you do not use then & PDMS will be suspended until the program is closed.

```
!file = object file(|C:\temp\pmllib\forms\c2ex1.pmlfrm|)
syscom |c:/WINDOWS/notepad.exe $!file&|
```

3.3.3 Using the Standard File Browser

PDMS is supplied with a standard file browser that can be used to load forms. The best way to load the form is to use the !!fileBrowser function as the function arguments will initialise the browser. To show the browser form, type the following:

```
!!fileBrowser('c:\','*.*','Example',false,'q var !!fileBrowser.file')
```

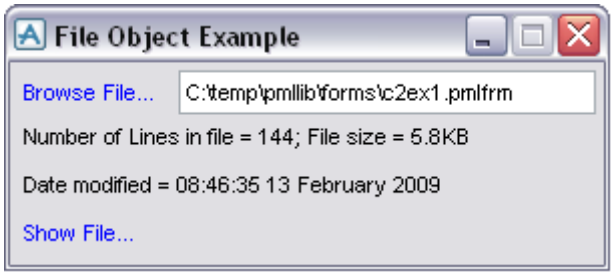
Where the first argument is the initial file path; the second is the file type filter; the third is the title for the browser form; the fourth is if the file needs to exist (true or false – used to save files); the fifth argument is the callback on the 'Open' button.

Navigate to a file and press open, a file object should be printed to the command window:

```
<FILE> C:\temp\pmllib\forms\c2ex1.pmlfrm
```

This shows that once a file has been chosen, it is held as the **.file** member of the fileBrowser form and is available for use. Show the example by typing **`show !!exampleFile`**. The example demonstrates how the object can be used to read a file, gain information about it and even show it.

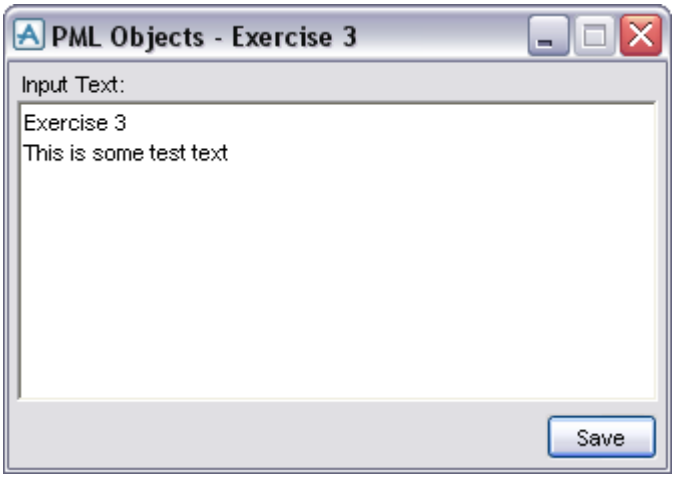
```
setup form !!exampleFile
  !this.formTitle = |File Object Example|
  button .browse linklabel |Browse File...| wid 8
  text .fileName || call |!this.read(object file(!this.txt1.val), 1)| width 25 is string
  para .par1 at x 0 ymax text || width 35
  para .par2 at x 0 ymax text || width 35
  button .load linklabel |Show File...| at x 0 ymax call |!this.load()| wid 8
  member .file is FILE
exit

define method .exampleFile()
  !this.browse.callback = |!!fileBrowser('C:\temp\pmllib\forms', '*.pmlfrm', $
  'Load File', TRUE, '!!exampleFile.read(!!fileBrowser.file, 2)')|
  !this.load.visible = FALSE
endmethod

define method .read(!file is file, !flag is REAL)
  !this.file = !file
  if !flag.eq(2) then
    !this.fileName.val = !file.string()
  endif
  !file.open('READ')
  !n = 0
  do
    !line = !file.ReadRecord()
    if !line.unset() then
      BREAK
    else
      !n = !n + 1
    endif
  enddo
  !file.close()
  !this.load.visible = TRUE
  !date  = !file.DTM()
  !size = (!file.size() / 1000)
  !this.par1.val = |Number of Lines in file = | & !n & |; File size = | $
  & !size.string(|d1|) & |KB|
  !this.par2.val = |Date modified = | & !date
endmethod

define method .load()
  !file = !this.file
  syscom |notepad.exe $!file&|
endmethod
```



Exercise 3 – Using the FILE Object

1

- Create a new form called !!c2ex3. The purpose of this form will be to use the FILE object and standard file browser to save any text written written in a textpane.
- The form will require a textpane and one button.
- The FILE object should be a member of the form.
- The save button shoud invoke the standard file browser.
- The only file type allowed should be .txt.



2

- Now modify the form so that the file browser opens in the last save location with the last saved name.
- This will require looking at the FILE objects methods.

An example of the completed form can be found in Appendix B

4 Collections

A very powerful feature of the PDMS database is the ability to collect and evaluate data according to rules. There are two available methods for collection that are valid in PML. The first is a command syntax PML 1 style and the other is with a PML **COLLECTION** object. Both are still valid, but when developing new PML, a **COLLECTION** object should be used in preference,

4.1 COLLECT Command Syntax (PML 1 Style)

The **COLLECT** syntax is based around three specific pieces of information:

- What element type is required?
- If specific elements are required, what is different about them?
- Which part of the hierarchy to look it?

If you wish to collect all the EQUI elements for the current ZONE, type the following:

```
var !equipment collect all EQUI for ZONE
q var !equipment
```

If you wish to collect all the piping components owned by a specific BRAN, type the following:

```
var !pipeComponents collect ALL with owner eq /200-B-4-B1 for SITE
q var !pipeComponents
```

if you wish to collect all the BOX primitives below the current element, type the following:

```
var !boxes collect all BOX for ce
q var !boxes
```

ⓘ *You do not need to specify level of the hierarchy to search within.*

📖 *For more examples, refer to Chapter 11 of the Database Management Reference Manual and 2.3.10 Design Reference manual general commands*

4.2 EVALUATE Command Syntax (PML 1 Style)

After elements have been collected through the **COLLECT** syntax, they are stored as an ARRAY. This array (like any array) can be processed using the **EVALUATE** syntax to provide further information

To get the names of all the elements held in **!equipment**, type the following:

```
var !equipmentNames evaluate NAME for ALL from !equipment
q var !equipmentNames
```

To get the fullnames of the elbows held within **!pipeComponents**, type the following:

```
var !elbows evaluate FLNN for all ELBO from !pipeComponents
q var !elbows
```

To get the names (without the leading slash) of the pumps in **!equipment**, type the following:

```
var !pumps evaluate NAMN for ALL with MATCHWILD(NAMN, |P*|) from !equipment
q var !pumps
```

To get the volume of all the boxes in **!boxes**, type the following:

```
var !volume eval (xlen * ylen * zlen) for ALL from !box
q var !volume
```

ⓘ *Notice how the expressions are command syntax and must return a BOOLEAN. Refer to the Software Customisation Reference Manual for more examples*

4.3 COLLECTION Object (PML 2 Style)

A **COLLECTION** object is another example of a PML object that has directly replaced some command syntax. It is recommended that all new PML uses **COLLECTION** objects as required. One advantage of using a **COLLECTION** object is that an **ARRAY OF DBREF** objects is returned.

A **COLLECTION** object is assigned to a variable in the same way as other objects:

```
!collection = object COLLECTION()
q var !collection
q var !collection.methods()
```

Once assigned, the methods on the object are used to set up the parameters of the collection. To set the element type for the collection to only **EQUI** elements, type the following:

```
!collection.type(|EQUI|)
```

If more than one element type is required, the following could be typed:

```
!elementTypes = |EQUI BRAN SCTN|
!collection.type(!elementTypes.split())
```

The scope of the collection must be a **DBREF** object and should be passed as an argument to the **.scope()** method. For example, type the following:

```
!collection.scope(!!ce)
```

To filter the results, the **.filter()** method must be passed an **EXPRESSION** object. This means that the process is in two steps:

```
!expression = object EXPRESSION(|name of owner eq '/200-B-4-B1'|)
!collection.filter(!expression)
```

Once the collection object has been set up, the **.results()** is used to return the collected elements as an **ARRAY of DBREF** objects.

```
!results = !collection.results()
q var !results
```

4.4 Evaluating the Results From a COLLECTION Object

After an **ARRAY of DBREF** objects has been generated from a **COLLECTION** object, the entire array can be evaluated using the **.evaluate()** method on an array object. The argument for the **.evaluate()** method must be a **BLOCK** object defined with the expression that needs evaluating.

To get an **ARRAY of STRINGs** holding the full names of the collected elements, type the following:

```
!block = object BLOCK(|!results[!evalIndex].flnn||)
!resultNames = !results.evaluate(!block)
q var !resultNames
```

Notice how the **BLOCK** object uses the local variable **!evalIndex**. This variable effectively allows the **.evaluate()** method to loop through the ARRAY. To get the positions of the collected elements, type the following:

```
!resultPos = !results.evaluate(object BLOCK(|!results[!evalIndex].pos|))
q var !resultPos
```

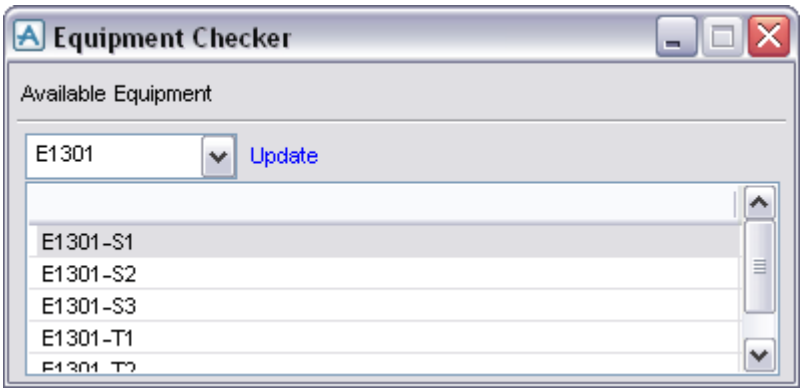
Instead of defining the block as a separate variable, this second example shows that the object can be defined within the argument to another object.

ⓘ *Notice how the evaluated ARRAY contains the correct object types generated from the evaluation*

Exercise 4 – Equipment Collections

1

- Create a new form that will act as an **Equipment Checker**. The purpose of the form is to provide users information about **EQUI** elements from the hierarchy
- The form shall use an **OPTION** gadget to display to the user the available EQUI elements and a **LIST** gadget to display the **NOZZ** elements owned by the chosen EQUI. Layout the form as the below screenshot.

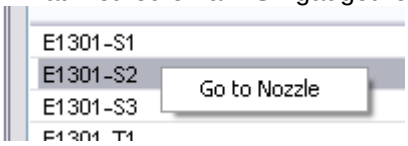


2

- Write a method that collects all the **EQUI** elements for the **ZONE** of the **CE**. Try and write the collection part of the method in a PML 1 style. What happens if no pieces of equipment are found in the **ZONE**? Call this method when the form is shown.
- Add an **'Update'** button that will re-run this method if a different **ZONE** is chosen
- Write a method that runs after the piece of equipment has been chosen. This method should collect all the **NOZZ** elements for that **EQUI** element. Try writing the collection part of the method in a PML 2 style. After the method has run, set the full names of the collected **NOZZ** elements to the **LIST**. Make use of the **RTEXT** and **DTEXT** members of the gadgets (to store names and **DBREFs**)

3

- Add a popup menu to the **LIST** gadget to allow the users to navigate to the chosen element (set the current element). What method on a **LIST** gadget retrieves the chosen **RTEXT**?

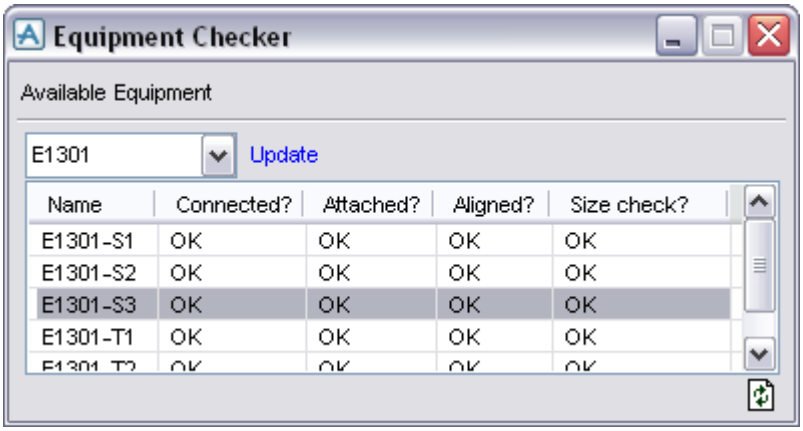


4

- Write a method that looks at each of the collected **NOZZ** element and checks the following:

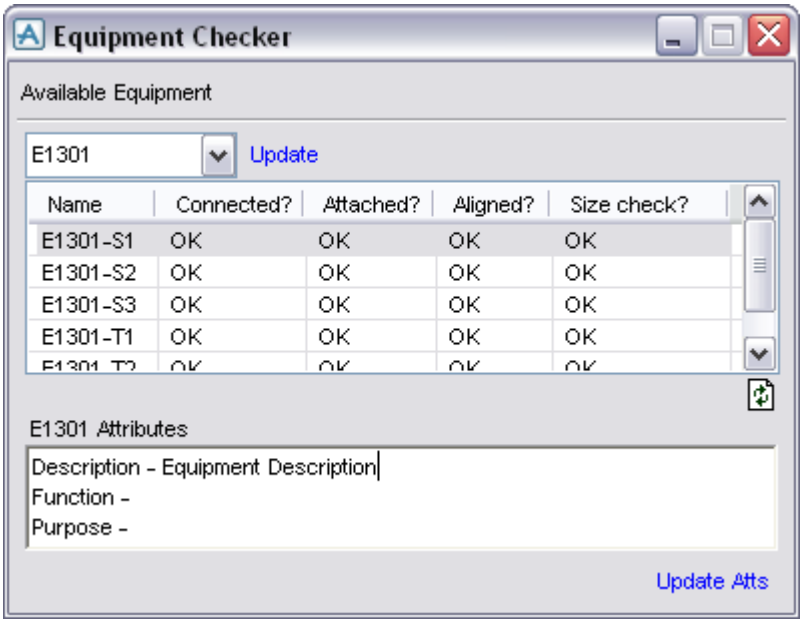
Is it connected?	This can be decided by checking the NOZZ element's CREF attribute. If the attribute is unset or the reference is invalid, then the user will have to check that nozzle.
Is it attached?	Check the positions of the NOZZ and PIPE elements and if they are different, then the NOZZ should be checked
Is it aligned?	Check the directions of the NOZZ and PIPE elements. If they are different, then they should be checked
Is it sized correctly?	Compare the diameter of the PIPE to the size of NOZZ. If they are different, then they should be checked.

- Display the results of the check in the LIST gadget using columns.
- Provide an Refresh button to recheck the nozzles incase the users changes anything in response to the check. Why not use the refresh icon from standard PDMS?



5

- Add a **TEXTPANE** to the form to allow users to enter attribute values for the chosen piece of equipment. The TEXTPANE should initially be filled with three attributes (**Description, Function and Purpose**) and the values for the chosen equipment. As the user chooses other pieces of equipment, the displayed attribute values should update
- Write a method to read the attributes and values from the **TEXTPANE**. This will involve splitting the information entered by the user. If it is a valid value and attribute, the attribute should be set.
- This method should cope with new attributes being entered by the user. Consider what should happen if an invalid attribute or value is entered.



- As the **TEXTPANE** has no callback, provide a button to call this method.

An example of the completed form can be found in Appendix B

5 View Gadgets

VIEW gadgets are named gadgets which are used to display the information from available databases. The way the information is displayed depends on the VIEW gadget type.

General View Types:

ALPHA views for displaying text output and / or allowing command input.
PLOT views for displaying non-interactive 2D plotfiles.

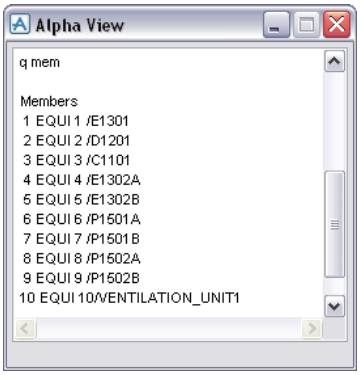
Application-specific View Types

AREA views for displaying interactive 2D graphical views.
VOLUME views for displaying interactive 3D graphical views.

Refer to the correct VIEW element type in the PDMS Customisation Reference Manual for more information

5.1 Alpha Views

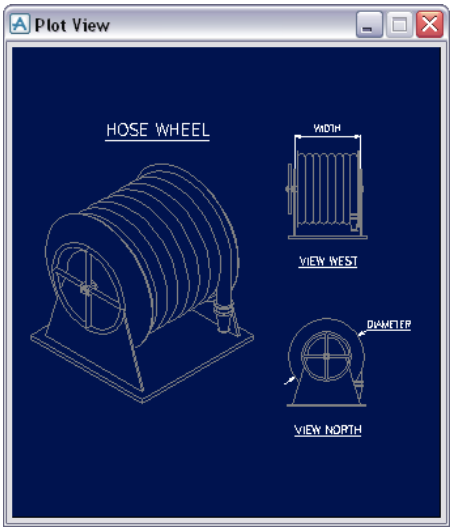
An **ALPHA** view gadget is the same that is used for the standard command window. To show the Alpha View example, type **show !!exampleAlphaView**:




```
setup form !!exampleAlphaView
  !this.formTitle = |Alpha View|
  view .input AT X 0 Y 0 ALPHA
    height 15 width 30
    channel REQUESTS
    channel COMMANDS
  exit
exit
```

5.2 Plot View Example

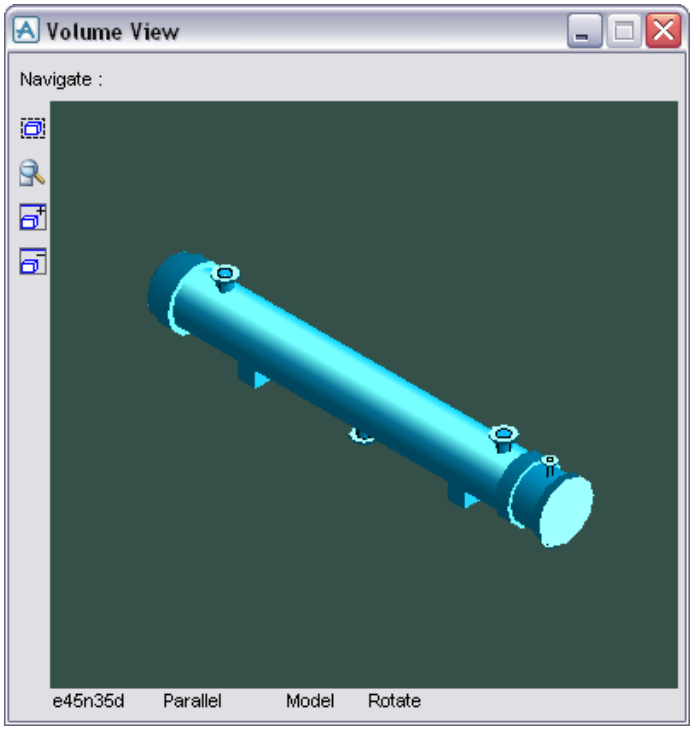
A **PLOT** view can be used to provide the user with extra information. The extra information would be contained in a **.plt** file and is applied to the view during the constructor method. To show the Plot View example, type **show !!examplePlotView:**




```
setup form !!examplePlotView
  !this.formTitle = |Plot View|
  view .plot plot width 41 hei 15
    CURS NOCURSOR
  exit
exit

define method .examplePlotView()
  !this.plot.borders = FALSE
  !this.plot.add(|/%PMLLIB%\icon\hosewheel.plt|)
endmethod
```

5.3 Volume View Example

A **VOLUME** view can be used to see the 3D data held within the database. The main PDMS window is an example of a volume view. To show the Volume View example, type **show !!exampleVolumeView**:



ⓘ *The code that supports this example is in Appendix B*

It is now possible in PDMS to assign different drawlists to different views. This has been demonstrated by giving the volume view in this example a different drawlist to the main view. This is possible through the use of two standard PDMS objects: **!!gphviews** and **!!gphdrawlists**

To investigate the **!!gphdrawlists**, type the following into the command window:

```
q var !!gphDrawlists
<GPHDRAWLISTS> GPHDRAWLISTS
   DRAWLISTS <ARRAY> 1 Elements
q var !!gphDrawlists.drawlists
<ARRAY>
   [1]  <DRAWLIST> DRAWLIST
q var !!gphDrawlists.methods()
```

The following are some examples of the methods which are available on **!!gphDrawlists**. To find out information about all the drawlists in the global object, use:

```
!!gphDrawlists.listall()
```

To create a drawlist object in the global object, use:

```
!!gphDrawlists.createDrawlist()
```

To associate a drawlist from the global object with a view gadget, use:

```
!!gphDrawlists.attachView(DRAWLIST, GADGET)
```

Drawlists can only be attached to a view that has been registered with the **!!gphViews** object. To investigate the !!gphViews, type the following into the command window:

```
q var !!gphViews
```

```
<GPHVIEWS> GPHVIEWS
   ACTIVEVIEW <GADGET> Unset
   SELECTEDVIEWS <ARRAY> 0 Elements
   VIEWS <ARRAY> 1 Elements
q var !!gphViews.methods()
```

To add a view to the global object, use:

```
!!gphViews.add(GADGET)
```

To set the limits of a view based on a DBREF object, use:

```
!!gphViews.limits(GADGET, DBREF)
```

The example volume view form mimics the functionality of the main 3D window by providing the following four buttons:

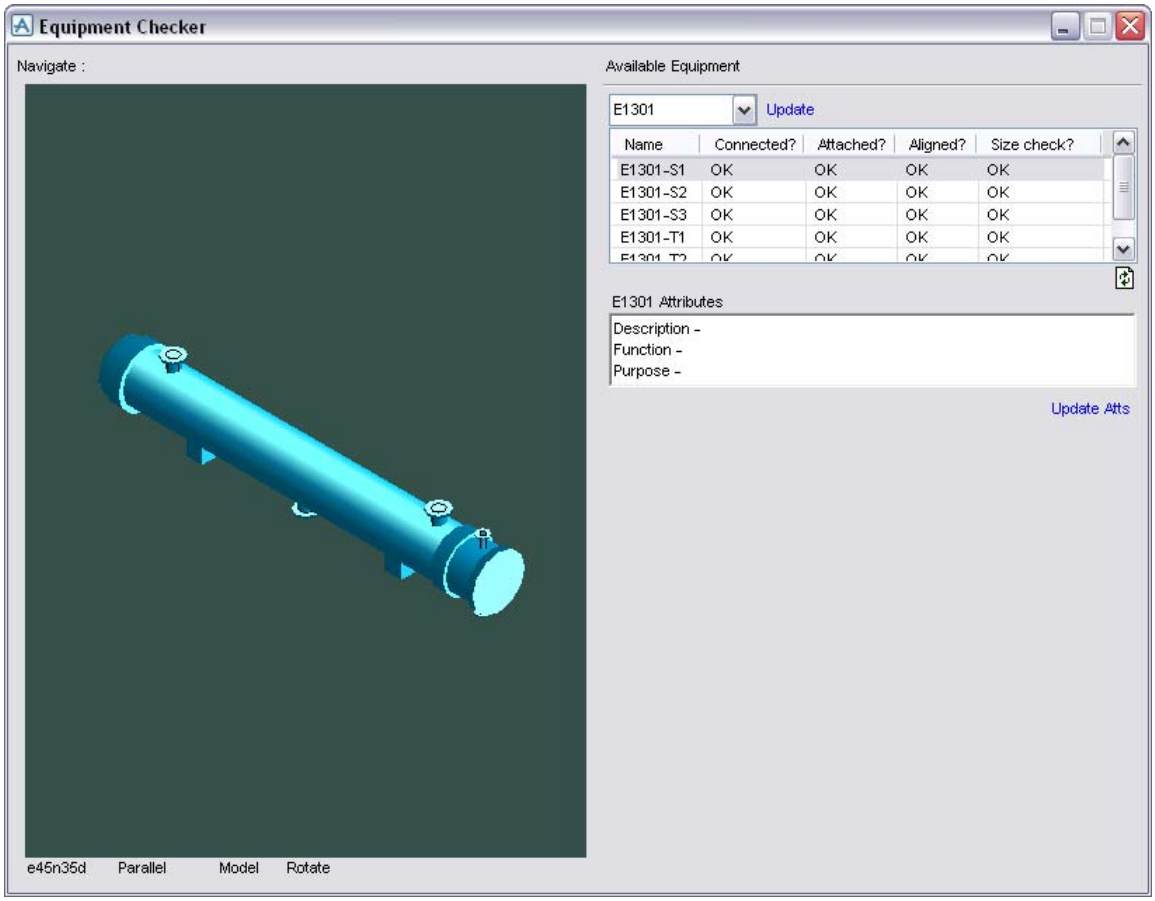
-  set view limits to CE
-  Walk to Drawlist (set limits of the view based on drawlist)
-  Add the current element to the drawlist
-  Remove the current element from the drawlist

These buttons are not always necessary when defining a volume view element, but have been included in the example to demonstrate some further methods on the object. Review form definition and its method to understand how the process works.

Exercise 5 – Adding a Volume View

1

- Add a **VOLUME VIEW** element to the form created in Exercise 4. The Volume View should have its own drawlist and should correctly display the chosen piece of equipment only
- Look at **!!exampleVolumeView** and see which parts of the PML can be re-used. How will the method know which element has been chosen and how will the drawlist/limits be updated?



2

- Add an **'Available Tasks'** section to the form. As functionality is added to the form, it shall be added here.
- Write a method that adds any connected elements to the drawlist (a **NOZZ** is connected through its **CREF** attribute). Elements should be added and removed through a toggle style linklabel button (with a pixmap – see screenshots)

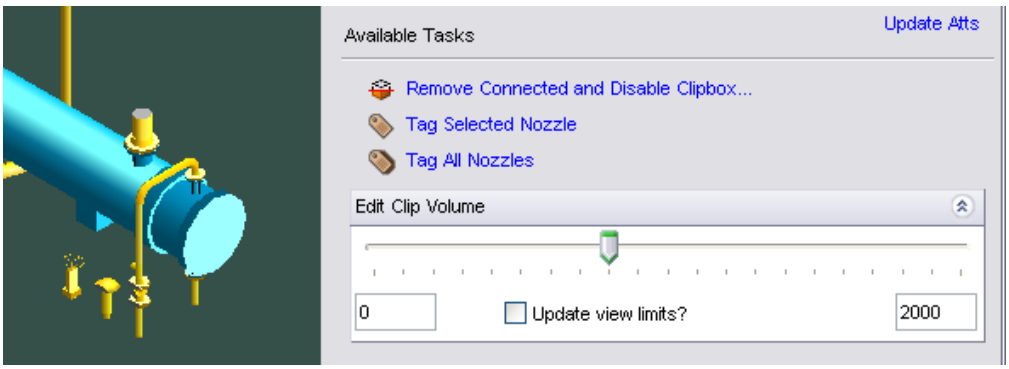
3

- Add a method to the form to tag **NOZZ** elements on the chosen piece of equipment. This will allow to the user to identify which nozzle is which.
- Provide two extra tasks: **Tag Selected Nozzle** (chosen in the list) and **Tag All Nozzles**

- When the user tags a nozzle, use the **MARK** or **AID TEXT** syntax to put the name of the nozzle into the volume view. The method should keep track of which nozzles are tagged so that they are only tagged once (use a form member?). This will also manage the state of the toggle so that it is always correct.

4

- Update the **'Add Connected'** method to include a clipbox. This will limit how much the user sees. A clipbox is added to a Volume View by using a **GPHCLIPBOX** object. The **VIEW** member of a **GPHCLIPBOX** object holds the name of the Volume View you wish to clip.
- The **GPHCLIPBOX** object should be stored as a member of the form. This is so it can be resized as different elements are chosen. To find out more about the **GPHCLIPBOX** object, open **gphclipbox.pmlobj**
- When the user turns the clipbox on, they should then see a hidden fold-up panel which holds a slider. This slider will allow the user to dynamically change the size of the clipbox. Make use of the **.refresh()** method to update the volume view during the slide

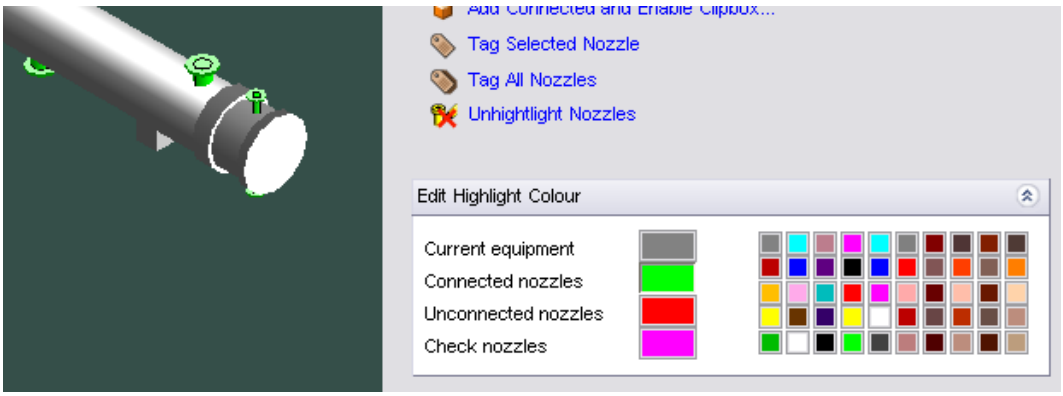


- Add a method to update the range of the slider based on the input into two text boxes. This will be in case the equipment is too large/small for the default value.
- Add a method that will update the limits of the view when the clipbox is resized. This is to stop the clipbox becoming larger than the limits of the view.

5

- Add a method to the form that will add colour to the elements of the drawlist to indicate the results of the nozzle check. Colour can be added through the **.highlight()** method on a **DRAWLIST** object.
- Three different colours should be added. A colour for unconnected nozzles, a colour for nozzles that need checking and a colour for connected nozzles which pass the checks. You may also wish to highlight the chosen piece of equipment.

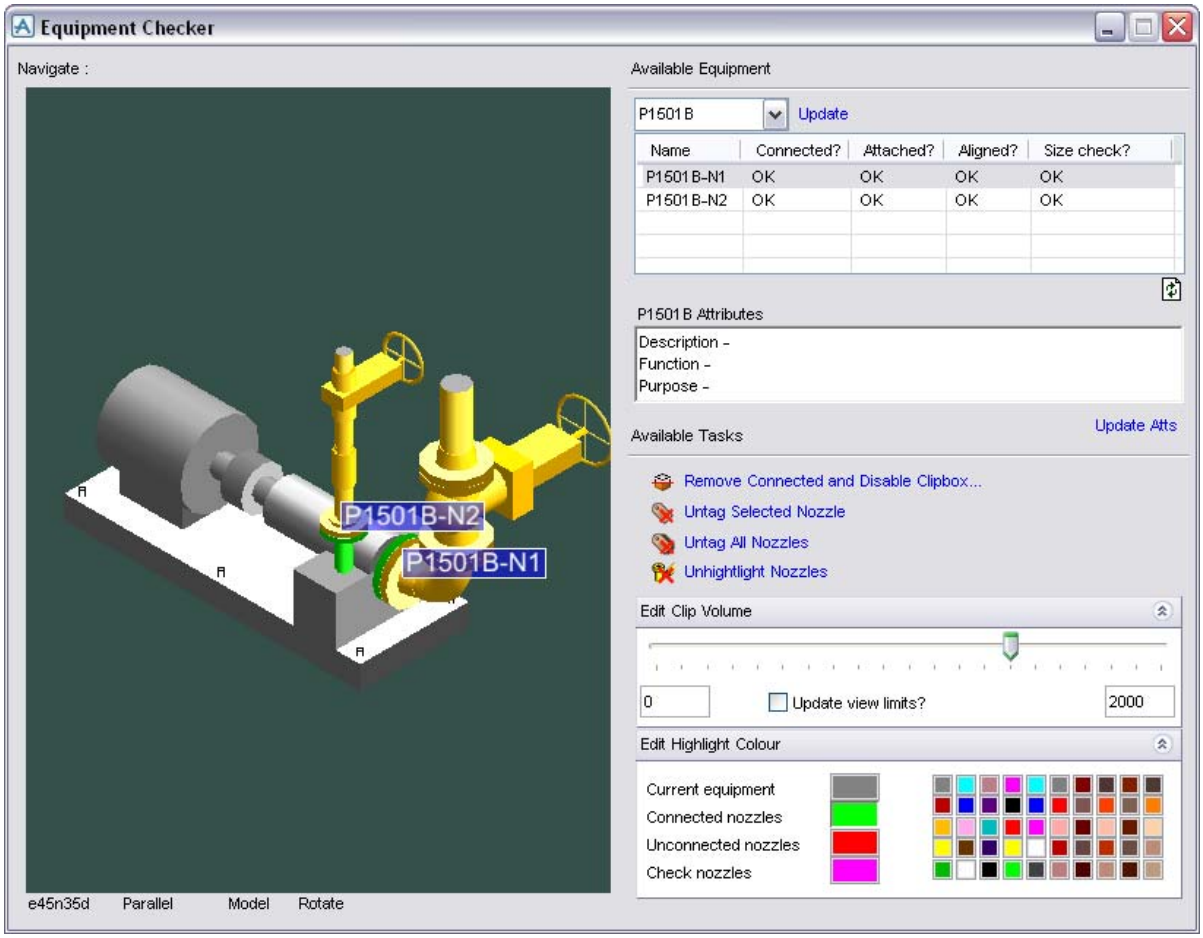
- When the user turns on the highlighting, show a hidden frame which allows users to change the colours. Four buttons shall display the current colours. On pressing on of those buttons, a hidden panel frame shall be shown which will allow the users to update the colour. Think about how a **DO** loop could be used to create this grid of buttons buttons.



- The methods will need to manage which elements are coloured, which colours should be used and the visibility of the gadgets on the form

6

- The form is now complete. Test it as a complete form and solve any errors that occur during this process.



An example of the completed form can be found in Appendix B

6 Event Driven Graphics (EDG)

The EDG has been developed to allow a common interface for appware developers to use when setting up the graphic canvas for graphical selection. This method is relatively simple and easily extendible. This will allow the developer to concentrate on the development of their own application without the need to know the underlying mechanism (core implementation) of the EDG interaction handlers and system.

The system handles all the underlying maintenance of the current events stacked e.g. associated forms, initialisation sequences, close sequences, etc.

The current implementation of the system has mainly been developed for interaction with the 3D graphic views in the Design module. However, the interface can be used with any of the modules that use the same executable and the standard 3D graphic views. It is intended that EDG will supersede the old ID@ syntax. When setting up an EDG event, the following should be considered:

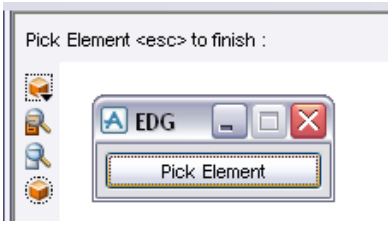
- What items will the user be picking?
- How many picks are required and in what order?
- What happens when the user makes a pick?
- What happens when the user has finished picking?

These aspects are controlled by methods on the objects associated with EDG. The main object is an **EDGPACKET** object. Once setup, this object is added to the **!!EDGCntrl** object that will activate the event.

ⓘ *For more information about EDG, refer to the EDG interface manual*

6.1 A Simple EDG Event

To demonstrate a simple EDG setup, show the example by typing show !!exampleSimpleEDG:




```
setup form !!exampleSimpleEDG
  !this.formTitle = |EDG|
  button .but1 | Pick Element | call |!this.pick()|
exit

define method .pick()
  -- Define event packet object
  !packet = object EDGPACKET()
  -- Define object as a predefined standard element pick
  !packet.elementPick(|Pick Element <esc> to finish|)
  -- Query the item member of the returned information from the pick
  !packet.action = |!!exampleSimpleEDG.process(!this.return[1])|
  -- Set what happens when the user presses esc
  !packet.close = |$p Finished|
  -- Add the event packet to the global EDG control object
  !!EDGCntrl.add(!packet)
endmethod

define method .process(!item is ANY)
  q var !item
endmethod
```

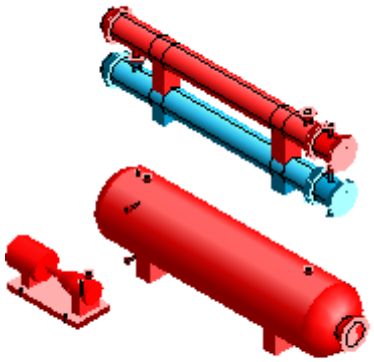
The example sets up a predefined element pick event by running the **.elementPick()** method on a **EDGPACKET** object. The subsequent methods specify what happens when a pick is made and what happens when the event is finished.

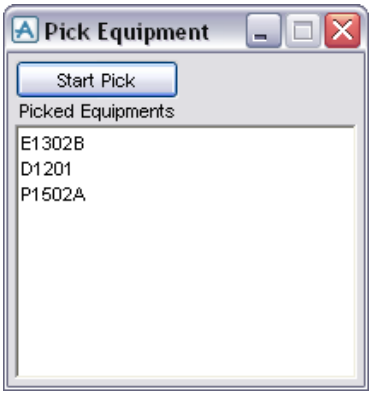
ⓘ *Notice how the action method of the EDGPACKET object references the form method explicitly. It displays the returned information (described in EDG interface manual)*

6.2 Using EDG

The following example shows how EDG can be applied to a form to provide picking functionality for users. The show the example, type show !!exampleEDG:

```
setup form !!exampleEDG
  !this.formTitle = |Pick Equipment|
  !this.initcall  = |!this.init()|
  !this.quitcall  = |!this.clear()|
  button .pick |Start Pick| call |!this.init()|
  textpane .txt1 |Picked Equipments| at x 0 ymax width 25 height 10
  member .storage is array
exit

define method .init()
  !this.clear()
  !packet = object EDGPACKET()
  !packet.elementPick(|Pick Equipments <esc> to add to list|)
  !packet.description = |Identify Equipment|
  !packet.action = |!!exampleEDG.identify(!this.return[1].item)|
  !packet.close = |!!exampleEDG.setInfo()|
  !!EDGCntrl.add(!packet)
endmethod

define method .identify(!pick is DBREF)
  do
    if !pick.type.eq(|EQUI|) or !pick.type.eq(|WORL|) then
      break
    else
      !pick = !pick.owner
    endif
  enddo
  if !pick.type.eq(|EQUI|) then
   if !this.storage.findfirst(!pick).unset() then
     !!gphDrawlists.drawlists[1].highlight(!pick, 314)
     !this.storage.append(!pick)
   else
     !this.storage.remove(!this.storage.findfirst(!pick))
     !!gphDrawlists.drawlists[1].unhighlight(!pick)
   endif
  endif
endmethod

define method .setInfo()
  !block = object BLOCK(|!this.storage[!evalIndex].flnn|)
  !names = !this.storage.evaluate(!block)
  !this.txt1.val = !names
endmethod

define method .clear()
  do !i values !this.storage
    !!gphDrawlists.drawlists[1].unhighlight(!i)
  enddo
  !!edgCntrl.remove(|Identify Equipment|)
  !this.txt1.clear()
  !this.storage.clear()
endmethod
```





This example shows how a method can take the returned information and use it. In this case, the picked element type is checked whether it is a piece of equipment. If it is not, the method loops up the hierarchy until one is found (or the world is reached). The EQUI is then highlighted if its new or unhighlighted if already chosen. There is a different method for the close action. For this reason, the picked elements are collected a form member. The close method applies the form member to the textpane gadget.

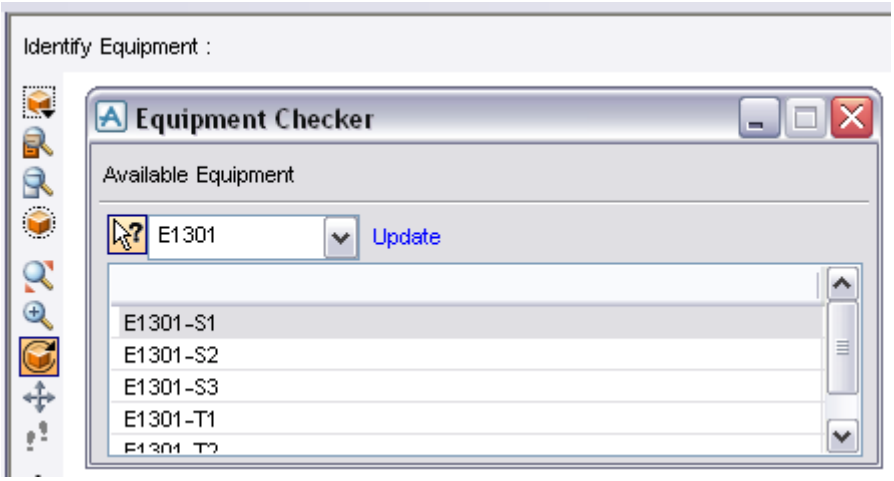
ⓘ *Notice how the form uses its **.clear()** method*

Exercise 6 – Adding EDG to Forms

1

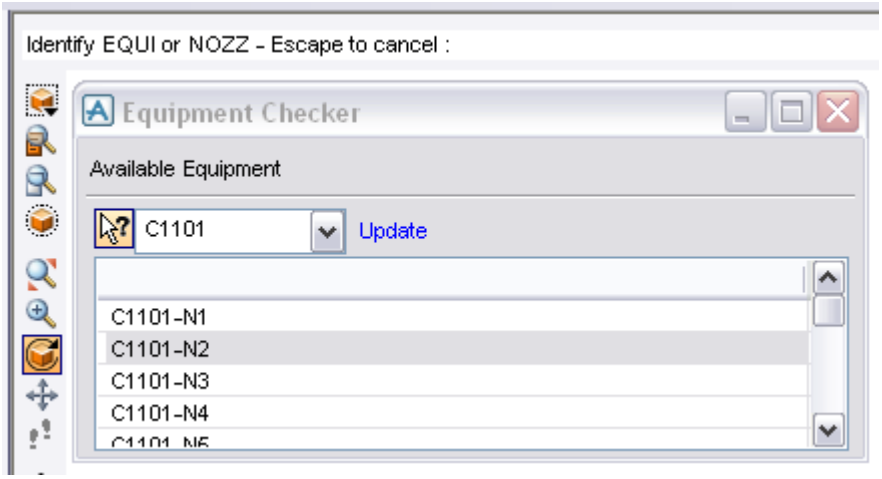
- Add a pixmap button to the form created in Exercise 4 that will allow users to pick which equipment they want from the main 3D window. Once picked, that piece of equipment (if available) should be selected in the **OPTION** gadget
- The button should initialise an EDG event. This EDG event should run a method on picking an element that does the following:

  - Turn the EDG event off after something has been chosen
  - Identify the type of element chosen
  - If an EQUI has been chosen, OK
  - Otherwise, loop up the hierarchy to find an EQUI.
  - If an EQUI cannot be found, report to the user and re-initialise the EDG
  - Find the chosen EQUI in the OPTION gadget and set it to the correct equipment
  - Run the nozzle collection method



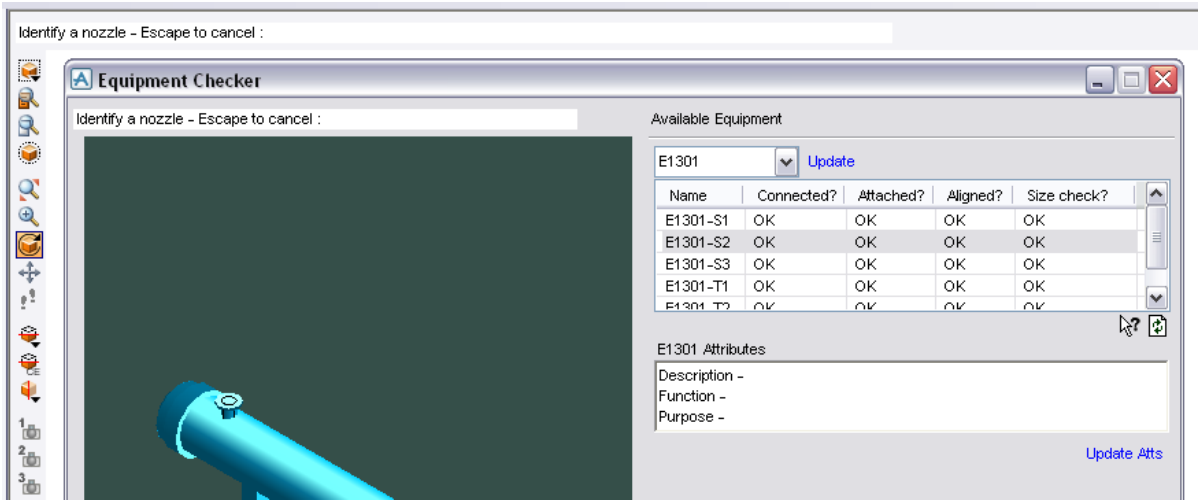
2

- Extend the method to allow **NOZZ** elements to be chosen (as well as EQUIs). When a nozzle is chosen, highlight it in the **LIST**. Consider what happens if a nozzle is chosen which is not in the list, but is owned by a piece of equipment in the **OPTION** gadget.



3

- Add an EDG event to form created in Exercise 5 that will allow the users to make their pick within the form's volume view. As the view will only contain one piece of equipment, the EDG should only be able to pick NOZZ elements.
- For this method to work, several **INMODEs** need to be applied to the volume view. Using show !!pmlforms, have a look at the volume view in the main **!!gph3Ddesign** form. Take a look at the function which defines this form (**gph3ddesign.pmlfnc)**



- The volume view on the form will need to be registered with the **!!EDGCntrl** object before it can be picked from. Use the **.addView()** and **.removeView()** methods on the **!!EDGCntrl** object to manage this.

An example of the completed form can be found in Appendix B

7 Miscellaneous

The following sections describe some other useful PML actions.

7.1 Error Tracing

It is possible to list all the commands that PDMS runs when a PML operation is carried out. This list can either be printed to the command window, or written to an external file.

Type $r109 /c:\temp\log.log onto the command line

Where c:\temp\log.log is the output file name (it shall be created if it does not exist)

Now every PML action will be written to the external file. The file will list every line of code and action carried out. Lines read will be indicated by a line number in square brackets. Lines not read will be indicated by a line number between round brackets. Entry and exit points between methods, functions and objects are indicated as well as any errors.

Type $r110 onto the command line to print the same lines to the command window.

ⓘ *Printing to the screen should only be done when a small number of lines are expected. The command window may run out of lines to display the information*

Once you are finished with error tracing, type $r to the command line. If the file is not closed, information will continue to be added to it

ⓘ *This will need to be typed before the output file can be opened*

📖 *To find out more information about he $r command, type $HR into the command window.*

7.2 PML Encryption

It is now possible to encrypt any files you create before sharing them using PML Publisher. Once encrypted, the files can still be used in any compatible AVEVA program, but they are not easily read through a normal text editor. Encrypted files may be used without additional licenses, but the encryption utility described below is separately distributed and licensed.

ⓘ *Once encrypted, PML cannot be decrypted. A reference copy of all PML should be kept safe*

The default encryption is implemented using the Microsoft Base Cryptographic Provider, which is included in, among other operating systems, Windows 2000 and Windows XP. There is a limit to the strength of this encryption and is not secure against all possible attacks. AVEVA may release improved algorithms with future releases of the product. If this happens, encrypted files would require re-encrypting.

Move the pmlencrypt.exe to the root folder where the files for encryption exist. Write a local .bat file containing the following line, where –pmllib is an instruction to encrypt PML files, ForEncrytion is a folder containing the source file and Encrypted is a folder where the encrypted files shall be put

```
pmlencrypt -pmllib ForEncryption Encrypted
```

If the example form !!nameCE was encrypted then the following file would be created:

```
--<004>-- Published PML 1.0 >--
return error 99 'Unable to decrypt file in this software version'
$** abdfe19b3008494b6399edda08b66004
$** MR+zhtg-egE2Ig9IiHSVmdPo08ChKexa7wbfcyODTbfjTFWU02pK3v4sXq5i
$** TKW3dEFRJCd60uSzaLXdc5fvLeOKqXO71uFlZ1vEsIOOHvq8viAwiys4rGXg
$** XLgFFVG7mpsmnFtrQDN3o51aiAgicFS6u08C7r8IaxUTUQAOdXeBmlp4TLXc
$** 9KR5LtAIugLrC9a7NxbF+0Hn-c5tOhUAEBG
```

📖 *Reading an encrypted file is slower than reading a decrypted on. Making use of the Buffer argument can help. Refer to the PML Publisher User Guide for more information*

8 Menu and Toolbar Additions

Previous to 11.6, if you required additions to the menus, you copied the **FSYSTEM/DBAR** files and manually edited them to add new menus.

Every time a new version of PDMS is released, you have to update any files you have copied.

As part of the .net work, this process has been removed so that no update will be needed in subsequent versions of PDMS. This process has been replaced with the new **ADDIN** File syntax.

To add new menus and toolbars to PDMS 11.6 or later, the **ADDIN** file syntax (**FSYSTEM/DBAR**) is no longer valid.

8.1 Miscellaneous Notes

To find out the menus currently contained within a model, use **q var !!appcatmain** for Paragon and **q var !!appdesmain** for Design.

More examples can be found in directory pmllib/design/objects. All the add-in application objects begin with the letters app****.pmlobj e.g. **appaccess.pmlobj**.

8.2 Modules that Use the New Addins Functionality

Addins are available in modules that have a form named appxxxmain.pmlfrm where xxx is the module such as des, cat, dra etc. These modules are Design (des), Draft (dra), Paragon (cat) and Spooler.

ⓘ *The spooler form is named spooler.pmlfrm but still uses addins.*

Isodraft currently uses the old FSYSTEM and cannot use addins. Admin and Monitor use separate forms named adminapplic.pmlfrm and monitormain.pmlfrm and also cannot use addins.

8.3 Adding a Menu/Toolbar

For a file created in **%PDMSUI%\DES\ADDINS** directory, the name doesn't matter, except for identification. The file will contain similar information to the old applications file in **%PDMSUI%**. This file is used for menu additions, new toolbars and standard application switching

8.4 Application Design

Below is an example of an ADDIN file which is used to load the Equipment application within design. Notice how the file only contains a number of file references. The addin file is automatically run by PDMS, so whatever this file references will also be run.

name:	EQUI	
directory:	EQUI	
showOnMenu:	TRUE	
object:	appEqui	
title:	Equipment	
callback:	CALLE XEQUIAPP	Macro to call when the menu is selected
synonym:	CALLE	Synonym to call equipment macros
startup:	$M /%PDMSUI%\DES\EQUI\INIT	Macro to run on startup of the EQUI application (for the first time)

8.5 Form and Toolbar Control

Forms are controlled using the APPFORMCNTRL object. Forms can be registered to:

- Appear only in certain applications
- Be reshown when PDMS restarts (serialisation)
- !!appFormCntrl.registerForm(!form is FORM)

Toolbars are controlled using the **APPTBARCNTRL** object: to avoid having too many toolbars on screen, each is enabled only when the user is in the right application. Users can add more forms/toolbars to these control objects, and those items will be shown to the user.

8.6 Converting Existing Applications

For users upgrading from the **FSYSTEM/DBAR** method, it is possible to transfer your existing files to the **ADDIN** syntax:

- Existing application has an entry in the PDMSUI%\DES\DFLTS\APPLICATIONS file
- New application has an add-in definition file in %PDMSUI%\DES\ADDINS

OLD	
# Equipment Application	
ApplicationDirectory:	EQUI
Title:	Equipment
Form:	_CDEQUIAPP
Callback:	CALLE XEQUIAPP
Synonym:	CALLE

NEW	
Name:	EQUI
ShowOnMenu:	TRUE
Directory:	EQUI
Title:	Equipment
Object:	APPEQUI
Callback:	CALLE XEQUIAPP
Synonym:	CALLE
Startup:	$M /%PDMSUI%\DES\EQUI\INIT

Previously, the main form was swapped when changing application. Existing applications defined menus and toolbars in the DBAR macro, which was called from the FSYSTEM macro. Now the main form stays the same and menus are shown and hidden as needed. Menus and toolbars are defined in an add-in object.

8.6.1 DBAR and Add-in Object

The contents of the ADD-IN object and the DBAR macro are very similar. The difference is the syntax used.

DBAR macro

```
$* Main Bar Definitions (Application specific)
label /BAR

   add |Position|   _POSITION
   add |Orientate| _ORI
   add |Connect|   _CONN

return

$* Create Pull Down Data
label /CREATE

   add sep
   add |Equipment...|      FORM _CDEQUI
   add |Sub-Equipment...| FORM _CDSUBEQ
   add |Primitives...|    FORM _CDPRIM
   add |Standard...|      form !!equCreateStd

return

$* Data for Form (Applicatio
label /DATA
exit

return

$* Menu Definitions (for application specific bar)
label /MENU

 menu _POSITION
   add |Explicitly (AT)...|          |!!pos3DExplicit()|
   add |Relatively (DY)...|
   add |Move|          MENU _MOVE
   add |Drag|          MENU _DRAG
   add |Plane Move|    MENU _PLMOVE
   add SEPARATOR
   add |Equipment Point| MENU _PPEAT
 exit

 menu _PPE
   add |At Explic...|  FORM _CDEPEX
   add |At General...| FORM _CDEPATG
   add |Through...|    FORM _CDEPMOVTH
   add |Clearance...|  FORM _CDEPM
   add |Towards...|    FORM _CDEPM

 exit

 menu _MOVE
```

ADD-IN object

```
define method .barMenu()
  !bmenu = object APPBARMENU()
  !bmenu.add(|Position|, 'POSITION')
  !bmenu.add(|Orientate|, 'ORI')
  !bmenu.add(|Connect|, 'CONN')
  !!appMenuCntrl.addBarMenu(!bmenu, 'EQUI')
endmethod
-- End of method definition for .barMenu()
------------------------------------------------------------------------
define method .createMenu()
  !menu = object APPMENU('sysCrt')
  !menu.add('SEPARATOR')
  !menu.add('FORM', |Equipment...|, 'CDEQUI', 'equiEquipment')
  !menu.add('FORM', |Sub-Equipment...|, 'CDSUBEQ', 'equiSubEquipment')
  !menu.add('FORM', |Primitives...|, 'CDPRIM', 'equiPrimitives')
  !menu.add('FORM', |Standard...|, 'equCreateStd', 'equiStandard')
  !!appMenuCntrl.addMenu(!menu, 'EQUI')
endmethod
-- End of method definition for .createMenu()

------------------------------------------------------------------------
define method .toolbars()
  frame .equiToolbar toolbar 'Equipment Toolbar'
    --define buttons
  exit
  !!appTbarCntrl.addToolbar('equiToolbar', 'EQUI')
endmethod
-- End of method definition for .toolbars()
------------------------------------------------------------------------
define method .menus()
  !menu = object APPMENU('POSITION')
  !menu.add('CALLBACK', |Explicitly (AT)...|, |!!pos3DExplicit()|, 'Explicitly')
  !menu.add('CALLBACK', |Relatively (BY)...|, |CALLIBD X3DPOSR|, 'Relatively')
  !menu.add('MENU', |Move|, 'MOVE', 'Move')
  !menu.add('MENU', |Drag|, 'DRAG', 'Drag')
  !menu.add('MENU', |Plane Move|, 'PLMOVE', 'PlaneMove')
  !menu.add('SEPARATOR')
  !menu.add('MENU', |Equipment Point|, 'PPEAT', 'EquipmentPoint')
  !!appMenuCntrl.addMenu(!menu, 'EQUI')

  !menu = object APPMENU('PPEAT')
  !menu.add('FORM', |At Explicit...|, 'CDEPEX', 'AtExplicit')
  !menu.add('FORM', |At General...|, 'CDEPATG', 'AtGeneral')
  !menu.add('FORM', |Through...|, 'CDEPMOVTH', 'Through')
  !menu.add('FORM', |Clearance...|, 'CDEPMOVCL', 'Clearance')
  !menu.add('FORM', |Towards...|, 'CDEPMOVTO', 'Towards')
  !!appMenuCntrl.addMenu(!menu, 'EQUI')

  !menu = object APPMENU('MOVE')
```

Add to bar menu

Add to Create menu

Define toolbars

Add new menus to form

8.6.2 Bar Menu and Toolbar

Add to bar menu

OLD

```
label /BAR
  add |Position|  _POSITION
  add |Orientate| _ORI
  add |Connect|   _CONN
return
```

NEW

```
define method .barMenu()
  !bmenu = object APPBARMENU()
  !bmenu.add('Position', 'POSITION')
  !bmenu.add('Orientate', 'ORI')
  !bmenu.add('Connect', 'CONN')
  !!appMenuCntrl.addBarMenu(!bmenu, 'EQUI')
endmethod
```

8.6.3 Define Toolbar

OLD

```
label /DATA
  -- define buttons
  -- for toolbar
return
```

NEW

```
define method .toolbars()
  frame   .equiToolbar   toolbar   'Equipment
     Toolbar'
    -- define buttons for toolbar
  exit
  !!appTbarCntrl.addToolbar('equiToolbar',
     'EQUI')
endmethod
```

8.6.4 Adding to Menus

OLD

```
label /CREATE
  add sep
  add |Equipment...|     FORM _CDEQUI
  add |Sub-Equipment...| FORM _CDSUBEQ
  add |Primitives...|    FORM _CDPRIM
  add |Standard...|  FORM !!equCreateStd
return
```

NEW

```
define method .createMenu()
  !menu = object APPMENU('sysCrt')
  !menu.add('SEPARATOR')
  !menu.add('FORM',  'Equipment...',  'CDEQUI',
     'equiEquipment')
  !menu.add('FORM',         'Sub-Equipment...',
     'CDSUBEQ', 'equiSubEquipment')
  !menu.add('FORM',      'Primitives...      ',
     'CDPRIM', 'equiPrimitives')
  !menu.add('FORM',       'Standard...       ',
     'equCreateStd', 'equiStandard')
  !!appMenuCntrl.addMenu(!menu, 'EQUI')
endmethod
```

8.6.5 Adding New Menus to Form

OLD

```
label /MENU

menu _POSITION
  add |Explicitly (AT)...|
    |!!pos3DExplicit()|
  add   |Relatively   (BY)...|   |CALLIBD
    X3DPOSR|
  add |Move|            MENU _MOVE
  add |Drag|            MENU _DRAG
  add |Plane Move|      MENU _PLMOVE
  add SEPARATOR
  add |Equipment Point| MENU _PPEAT
 exit
 …
return
```

NEW

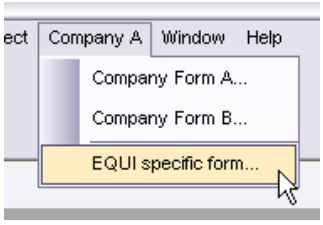
```
define method .menus()
  !menu = object APPMENU('POSITION')
  !menu.add('CALLBACK',  |Explicitly  (AT)...|,
     |!!pos3DExplicit()|, 'Explicitly')
  !menu.add('CALLBACK',  |Relatively  (BY)...|,
     |CALLIBD X3DPOSR|, 'Relatively')
  !menu.add('MENU', |Move|, 'MOVE', 'Move')
  !menu.add('MENU', |Drag|, 'DRAG', 'Drag')
  !menu.add('MENU',  |Plane  Move|,  'PLMOVE',
     'PlaneMove')
  !menu.add('SEPARATOR')
  !menu.add('MENU',     |Equipment     Point|,
     'PPEAT', 'EquipmentPoint')
  !!appMenuCntrl.addMenu(!menu, 'EQUI')
  …
endmethod
```

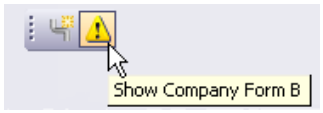
8.7 Example Object Including Toolbars, Bar Menus and Menus

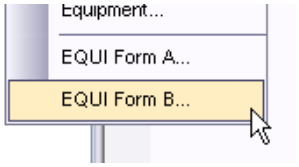
This is only an example showing adding toolbars, bar menus and menus. The effect of this object will be to add a new toolbar and main menu. It will also add some extra menu options to the EQUI application, including some under the Utilities menu. It will demonstrate how user-defined forms/functions/callbacks can be added to the PDMS menu system.

ⓘ *The methods .modifyForm() and .modifyMenus() run automatically, so can be used to call the objects methods.*

Type out the following and save it as appcompa.pmlobj








```
define object APPCOMPA
endobject

-------------------------------------------------------------------
-- Method:      .modifyForm()
-- Description: Standard method
--              Makes modifications to the main form
-------------------------------------------------------------------
define method .modifyForm()
  !this.toolbars()
endmethod

-------------------------------------------------------------------
-- Method:      .modifyMenus()
-- Description: Standard Method
--              Runs all other menu creation methods
-------------------------------------------------------------------
define method .modifyMenus()
 !this.barMenu()
 !this.menus()
 !this.specificMenus()
 !this.UtilsMenu()
endmethod

-------------------------------------------------------------------
-- Method:      .UtilsMenu()
-- Description: Adds to the Utilities menu in the EQUI application
-------------------------------------------------------------------
define method .UtilsMenu()
  !menu = object APPMENU('SYSUTIL')
  !menu.add('SEPARATOR')
  !menu.add('CALLBACK', |EQUI Form A...|, |$p Form A|)
  !menu.add('CALLBACK', |EQUI Form B...|, |$p Form B|)
  !!appMenuCntrl.addMenu(!menu, 'EQUI')
endmethod

-------------------------------------------------------------------
-- Method:      .toolbars()
-- Description: Creates toolbar for all applications
-------------------------------------------------------------------
define method .toolbars()
  frame .MyFormToolbar toolbar 'CompanyA Toolbar'
    !iconSize = !!comSysOpt.iconSize()
    !Pixmap1 = !!pml.getPathName('createpipe-' & !iconSize & '.png')
    !Pixmap2 = !!pml.getPathName('exclamation-' & !iconSize & '.png')
    button .but1 'Show Form A' tooltip 'Show Company Form A' $
      pixmap /$!<Pixmap1> width $!iconSize height $!iconSize callback |$p Form A|
    button .but2 'Show Form B' tooltip 'Show Company Form B' $
      pixmap /$!<Pixmap2> width $!iconSize height $!iconSize callback |$p Form B|
  exit
  !!appTbarCntrl.addToolbar('CompAToolbar', 'ALL')
endmethod
```

```
------------------------------------------------------------------
-- Method:      .barMenu()
-- Description: Adds options to the bar menu for all applications
------------------------------------------------------------------
define method .barMenu()
  !bmenu = object APPBARMENU()
  !bmenu.add(|Company A|, 'CompAMenu')
  !!appMenuCntrl.addBarMenu(!bmenu, 'ALL')
endmethod

------------------------------------------------------------------
-- Method:      .menus()
-- Description: Creates menus to the bar for all applications
------------------------------------------------------------------
define method .menus()
  !menu = object APPMENU('CompAMenu')
  !menu.add('CALLBACK', |Company Form A...|, |$p Form A|)
  !menu.add('CALLBACK', |Company Form B...|, |$p Form B|)
  !!appMenuCntrl.addMenu(!menu, 'ALL')
endmethod

------------------------------------------------------------------
-- Method:      .menus()
-- Description: Creates menus to the bar for specific applications
------------------------------------------------------------------
define method .specificMenus()
  !menu = object APPMENU('CompAMenu')
  !menu.add('SEPARATOR')
  !menu.add('CALLBACK', |EQUI specific form...|, |$p EQUI specific|)
  !!appMenuCntrl.addMenu(!menu, 'EQUI')
endmethod
```

To call the above object, copy out the following into a new file called **COMPA** (no file extension) and save it into **%PDMSUI%\des\addins**. Update %PDMSUI% in PDMS.bat if not done already.

```
name:         COMPA
title:        Company A app
showOnMenu:   FALSE
object:       APPCOMPA
```

Where: Name is the unique id for user and addin code; Title is the text description of the add-in; ShowOnMenu determines whether the add-in has an entry on the Applications menu, i.e. whether the add-in defines an application (false by default). If it were an application addin like MDS this would be TRUE; Object is the object file that is used to define the menu/toolbar additions.

These are some optional lines that can be included. **Callback** (typically CALLE XEQUI Macro to call when the menu is selected) **Startup** (typically $m/%pdmsui%/des/equi/init Macro to run on startup of the equi app (for first time)) **ModuleStartup** (the callback run when the PDMS module first starts) StartupModify (Name of application to modify and the callback run when an application is first started, separated by a colon.e.g. EQUI:!!equiAppStartupCall()) SwitchModify (Name of application to modify and the callback to run when the application is switched to, separated by a colon. e.g. PIPE:!!pipeAppSwitchCall())

To add a startup macro to more than one application, add more entries into the add-in file typically:-

```
switchModify:EQUI:!!equiAppStartupCall()
switchModify:PIPE:!!equiAppStartupCall()
```

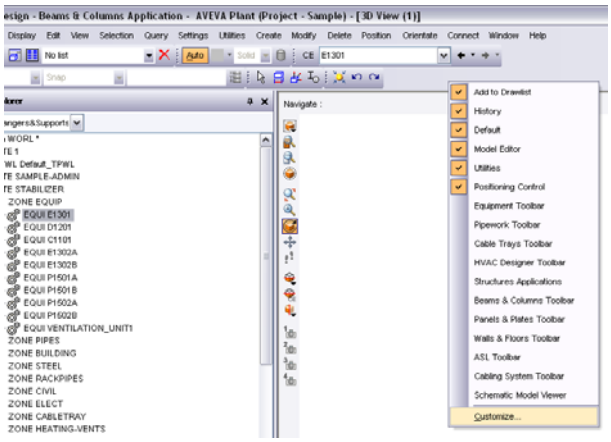
The following keys are only used for applications, i.e. add-ins that have a menu option on the Applications menu and can be switched to.

Menu	Entry for application on the applications menu (the title is used if this isn't specified)
Directory	Name of the application directory under %PDMSUI%\module
Synonym	Synonym created for the directory (only used if the directory is specified)
Group	Group number (applications with the same group number are put into a submenu together)
GroupName	Description of submenu to appear on main menu

It is possible to create toolbars and menus with a PDMS session by choosing Customize… from the available toolbars.

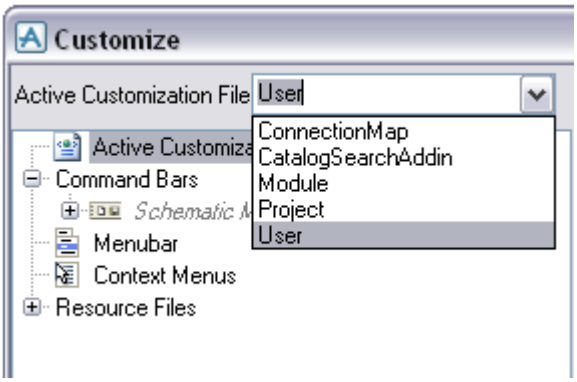
Worked Example – Creating a Toolbar Within Design

1



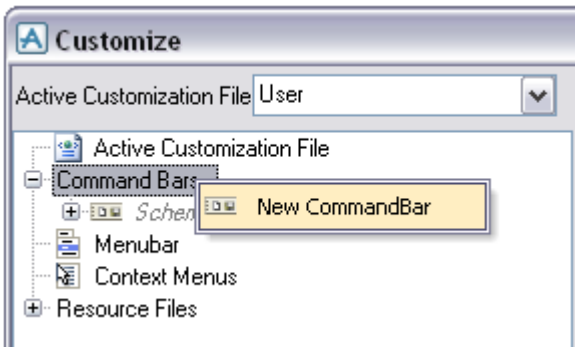
- Open the Customisation Form. Do this by right clicking on an empty part of the toolbar and choose Customize…

2



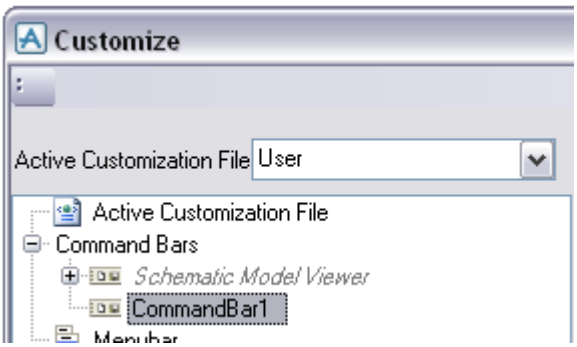
- On the Active Customization File, choose User.
- This will save an changes to a specific user file (file path given on the right hand side of the form)

3



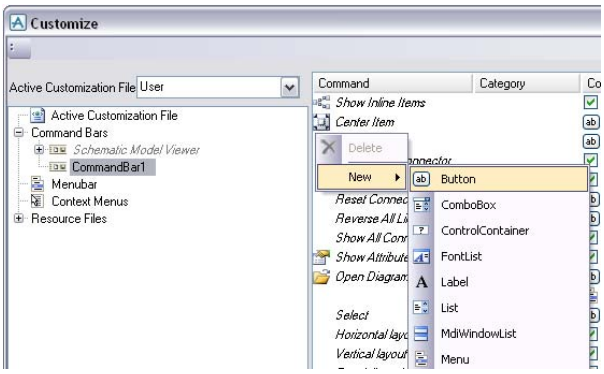
- Right click on the Command Bars heading in bottom window, and click on New CommandBar

4



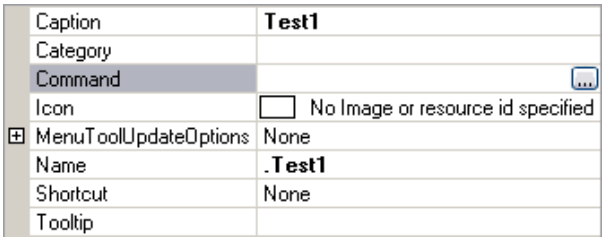
- You will now see a preview of the command bar in the top left of the form (when the new CommandBar is selected)

5



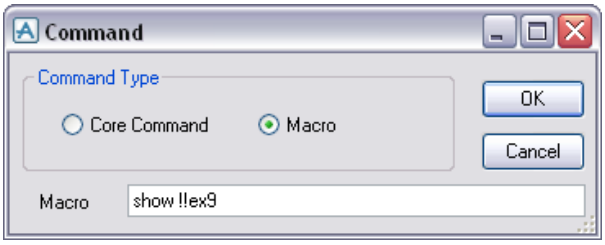
- Right click in the middle window in the form
- Create a new button object in the tools window

6



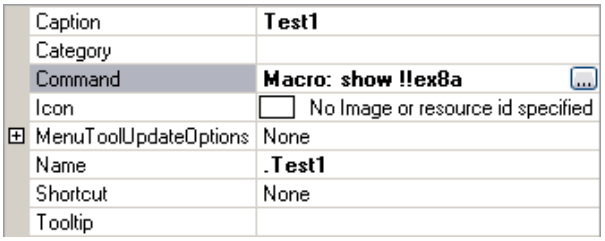
- With the new button selected, look to the right window on the form
- Select the line titled Command and click the small "…" button which appears on the right. This will let us set the command.

7



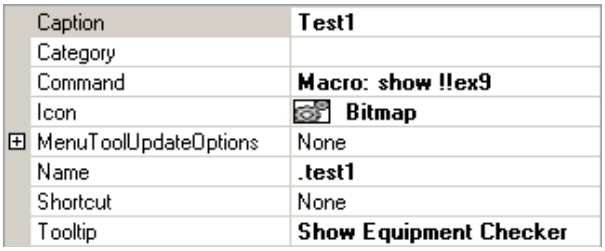
- A command can be a core command or a PML command.
- Choose Macro, and type **show !!ex9**. Close the form by pressing OK.

8



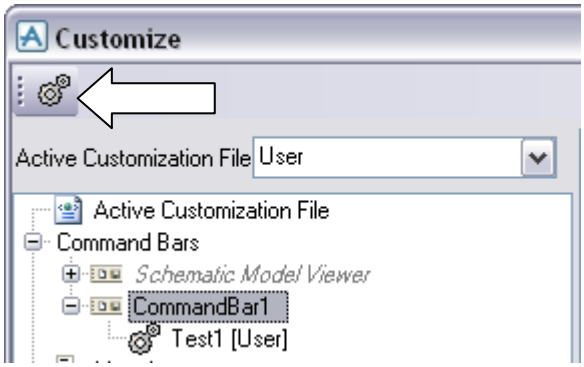
- The associated command is now
- Select the line titled **Icon**. Click the "…" button that appears to the right.
- Browse for the file equi.png (a supplied file)
- Click Open
- displayed in the properties window

9



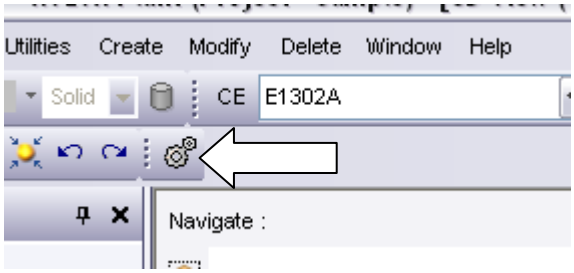
- The icon is now displayed in the right hand window.
- Set the tooltip to 'Show Nozzle Checker'

10



- Select the new button in the middle window.
- Drag and Drop the new button beneath the new CommandBar object to make the association.
- Select the CommandBar object and notice the button is displayed in the preview.

11



- Apply and OK the form and the new Command Bar will be displayed in the main application.
- Test the button

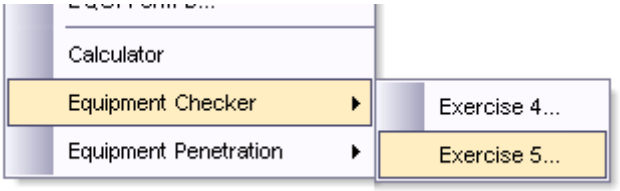
Exercise 7 – Add a Utility and Toolbar Menu

1

- Create some additional menus that will sit within the Equipment application and allow you access to your forms.
- Create an addin file that will point to the definition of the new menus. Call your file **PML** (with no file extension) and save it to **%pdmsui%/des/addins.** The addin should call apppml.pmlobj
- Once the files are saved, type **PML REHASH ALL** and the **DESIGN** to re-enter Design. Test the menus in a Design session

2

- Create a user toolbar using the Customize… option in Design. Use some of the supplied icons to create buttons that will show some the form created.
- Test the buttons



ⓘ *Please note that even a simply spelling mistake in the above exercise can prevent design from loading*

If Design fails to load, do the following:

- Read the error message in the command window and action appropriately
- If objects are “not found”, check the file paths and type **PML REHASH ALL**
- Once the error has been fixed, type **DESIGN** onto the command line and Design should load.
- If typing **DESIGN** generates an error, the command line is still in setup mode and needs exiting – type **EXIT**
- To close the command line, type **FINISH**

Appendix A – Summary of Grid Control Gadget Methods

The following table shows all the current API calls to the grid control gadget. These can be used in a PML form or on the PDMS command line:

Name	Type	Purpose
Grid Population		
NetDataSource(String TableName, Array Attributes, Array Items)		Data Source constructor for database Attributes and database items. The Grid will populate itself with database attribute values.
NetDataSource(String TableName, Array Attributes, Array AttributeTitles, Array Items)		Data Source constructor for database Attributes and database items. The Grid will populate itself with database attribute values. The attribute titles can be added.
NetDataSource(String TableName, Array columns, Array of Array of rows)		Data Source constructor for column headings, and a set of rows. In this case the grid does NOT populate itself with database attribute values.
NetDataSource(String TableName, Array Attributes, Array Items, String Tooltips)		Data Source constructor for database Attributes and database items. The Grid will populate itself with database attribute values.
NetDataSource(String TableName, Array columns, Array of Array of rows, String Tooltips)		Data Source constructor for column headings, and a set of rows. In this case the grid does NOT populate itself with databases attribute values.
NetDataSource(String TableName, string PathName)		Data Source constructor for import of an Excel XLS or CSV File
BindToDataSource(NetDataSource)		Bind Grid to NetDataSource
Grid Settings		
getDataSource()	NetDataSource	Returns the data source that has been set in the grid
SaveGridToExcel(String)		Save All Data to Excel File. Note that this will retain all grid groupings and layout of data. The String should specify the full pathname, eg: 'C:\temp\file.xls').
SaveGridToExcel(String, Worksheet)		Save All Data to a Worksheet in an Excel File. Note that this will retain all grid groupings and layout of data.
FixedHeaders(Boolean)		Allow/disallow fixed headers. (Used when scrolling).
FixedRows(Boolean)		Allow/disallow fixing of rows. (Used when scrolling).
OutlookGroupStyle(Boolean)		Allow/disallow Outlook group facility
HideGroupByBox(Boolean)		Hide the groupBy box….this will leave the Outlook style grouping in place and hence disallow the user from changing the grouping

setGroupByColumn(colNum, Boolean)		Set/unset column in OutlookGroupStyle
ColumnExcelFilter(Boolean)		Allow/disallow Excel style filter on columns
ColumnSummaries(Boolean)		Allow/disallow Average, Count, etc… on numeric columns
HeaderSort(Boolean)		Allow alphabetic column sorting
ErrorIcon(Boolean)		Allow/disallow the display of the red icon in the cell if a data value is not available
SingleRowSelection(Boolean)		Set grid to single row selection
allGridEvents(Boolean)		Switch Grid Events On or Off
ClearGrid()		Remove data & column headings.
RefreshTable()		For a grid displaying Dabacon data this recalculates the content of every cell to refresh to the latest database state.
setAlternateRowColor(String)		Set alternate rows to a different colour
setAlternateRowColor(Red Num, Green Num, Blue Num)		Set alternate rows to a different color. The default is: (251, 251, 255)
FeedbackSuccessColor(String)		Nominate (or query) the colour used to highlight cells successfully edited.
FeedbackFailColor(String)		Nominate (or query) the colour used to highlight cells where editing fails
SyntaxErrorColor(String)		Nominate (or query) the colour used to highlight cells containing syntax errors
ReadOnlyCellColor(String)		Nominate (or query) the colour used to highlight cells that cannot be edited.
resetCellFeedback()		Reset the cell editing feedback highlight colour and tooltip text.
DisplayUnsetAs(String)		Nominate (or query) a text string to be used in grid cells that hold “unset” attribute data.
DisplayNulrefAs(String)		Nominate (or query) a text string to be used in grid cells that hold “nulref” attribute data.
EditableGrid(Boolean)		Allow/disallow cells to be editable. Note that if the user is allowed to write data to the grid, then this data does not get written back to the database. The calling code is in control as to whether to either allow or disallow the change. This does not enable the user to add/remove rows, just to edit the content of the existing rows.
BulkEditableGrid(Boolean)		Allow/disallow cells to be bulk editable. Allows multiple cells to be selected, and Fill down/Fill up operations to be performed. Copy and Paste operations are also permitted to apply to multiple cells selected in the same column. The note about synchronising cell data with the data source as mentioned above for EditableGrid also applies to BulkEditableGrid.
setLabelVisibility(Boolean)		Show/Hide label

CreateValueList(Name, Array for List)		Create a value list for the grid
ResetColumnProperties()		Reset column layout
SaveLayoutToXml(String)		Save the grid layout (not data) to a file on the file system.
loadGridFromXml(String)		Load a stored layout on the file system (not data) into a grid instance.
PrintPreview()		Opens an Infragistics style Print Preview Form (which allows printing)
saveGridToExcel(string excel file, string worksheet, string strHeader)		Where strHeader is a user supplied string which is outputted to the first row in the spreadsheet. The rest starts at row 2.
Rows		
GetSelectedRows()	Array of Array (rows)	Returns array of selected rows
GetSelectedRowTags()	Array of row tags	Get the selected row tags
GetRows()	Array of Array (rows)	Returns array of rows
GetRow(NUM)	Array	Get the cells in the Row number=NUM
GetRow(row tag)	Array	Get the cells in the Row with the specified row tag
GetRowTag(NUM)	STRING	Get the Tag ID of the row
setEditableRow(Integer, Boolean)		Allow/disallow a designated row to be editable
setRowColor(NUM, String)		Set row color to a string (eg 'red')
setRowTooltip(row, String)		Set row tooltip to the specified string
setSelectedRowTags(Array rows)		Programmatically select the given row tags
selectRow(NUM)		Programmatically select the row=NUM
getNumberRows()	REAL	Get the number of rows in the grid
clearRowSelection()		Clear row selection
addRow(Array data)		Add a single row of data. If DB Element grid then only first array data is read. If not, then the Array data represents the data in each cell of the row,
deleteSelectedRows()		Delete selected rows
deleteRows(Array data)		Delete one or more rows given by the "Tags" in the array.
deleteRows(Array data)		Delete one or more rows given by the "Tags" in the array.
scrollSelectedRowToView(Boolean)		Scroll the selected row into view
setRowVisibility(double rowNum, Boolean)		Turn on/off the visibility of the specified row
IsRowVisible(double rowNum)		Query the visibility of the specified row
Columns		

SetColumnMask(NUM, STRING(MASK))		Set the Mask on column number=NUM (See valid masks in table below)
getColumnMask(NUM)	STRING	Get the mask on the specified column
setEditableColumn(Integer, Boolean)		Allow/disallow a designated column to be editable
GetNumberColumns()	REAL	Return the number of columns
AssignValueListToColumn(Name, Column NUM)		Assign the value list to a column
GetColumns()	Array of STRING	Get the columns in the grid
GetColumn(NUM)	Array	Get the cells in the Column number=NUM
setColumnColor(NUM, String)		Set column color to a string (eg 'red')
setColumnImage(NUM, String)		Set all cell images in column to the pathname of the specified icon
setColumnVisibility(colNum, Bool)		Show/Hide column
getColumnKey(NUM)	STRING	Get the Key of the column
setNameColumnImage()		Displays standard icons in a "NAME" attribute column when database items are used.
AutoFitColumns()		Resize columns to fit widest text
SetColumnWidth(Column NUM, Column Width)		Sey column number to a set width
ExtendLastColumn(Bool)		Extend the last column, or not
ResizeAllColumns()		Resize columns to fit in available width of form
IsColumnVisible(double colNum)		Query the visibility of the specified column.
Cells		
GetCell(ROW, COL)	STRING	Get the content of cell(ROW,COL) by row and column number
GetCell(row tag, column tag)	STRING	Get the content of cell(ROW,COL) by row tag and column tag
setCellColor(row, col, String)		Set cell color to a string (eg 'red')
setEditableCell(Row NUM, Col NUM, Boolean)		Allow/disallow a designated cell to be editable
setCellTooltip(row, col, string)		Set cell tooltip to the specifed string
setCellImage(row, col, String)		Set cell image to the pathname of the specified icon
setCellValue(rowNum, ColNum, STRING)		Programmatically set a cell value to a string
AssignValueListToCell(Name, Row NUM, Column NUM)		Assign the value list to a cell
DoDabaconCellUpdate(ARRAY args)		This function is provided to support user code in the BeforeCellUpdate event when the grid is displaying Dabacon element/attribute data. The

		array provided as argument to the BeforeCellUpdate event can simply be passed on to this function to modify the Dabacon element/attribute in synchronisation with the cell data. If the function is unable to perform the modification for any reason the Array[3] and Array[4] values will be set to indicate the problem.

Input masks can consist of the following characters:

Character	Description
#	Digit placeholder. Character must be numeric (0-9) and entry is required.
.	Decimal placeholder. The actual character used is the one specified as the decimal placeholder by the system's international settings. This character is treated as a literal for masking purposes.
,	Thousands separator. The actual character used is the one specified as the thousands separator by the system's international settings. This character is treated as a literal for masking purposes.
:	Time separator. The actual character used is the one specified as the time separator by the system's international settings. This character is treated as a literal for masking purposes
/	Date separator. The actual character used is the one specified as the date separator by the system's international settings. This character is treated as a literal for masking purposes.
****	Treat the next character in the mask string as a literal. This allows you to include the '#', '&', 'A', and '?' characters in the mask. This character is treated as a literal for masking purposes.
&	STRING
>	Convert all the characters that follow to uppercase.
<	Convert all the characters that follow to lowercase.
A	Alphanumeric character placeholder. For example: a-z, A-Z, or 0-9. Character entry is required.
a	Alphanumeric character placeholder. For example: a-z, A-Z, or 0-9. Character entry is not required.
9	Digit placeholder. Character must be numeric (0-9) but entry is not required.
-	Optional minus sign to indicate negative numbers. Must appear at the beginning of the mask string.
C	Character or space placeholder. Character entry is not required. This operates exactly like the '&' placeholder, and ensures compatibility with Microsoft Access.
?	Letter placeholder. For example: a-z or A-Z. Character entry is not required.
Literal	All other symbols are displayed as literals; that is, they appear as themselves.
n	Digit placeholder. A group of n's can be used to create a numeric section where numbers are entered from right to left. Character must be numeric (0-9) but entry is not required.
mm, dd, yy	Combination of these three special strings can be used to define a date mask. mm for month, dd for day, yy for two digit year and yyyy for four digit year. Examples: mm/dd/yyyy, yyyy/mm/dd, mm/yy.
hh, mm, ss, tt	Combination of these three special strings can be used to define a time mask. hh for hour, mm for minute, ss for second, and tt for AP/PM. Examples: hh:mm, hh:mm tt, hh:mm:ss.

Appendix B – Example Code

This appendix contains examples of code that provide solutions to each of the exercises. The whole code has not been included in this appendix. Only new/modified code is included so it will require the user to read and understand this code.

Appendix B1 - Example c2ex1.pmlfrm

```
setup form !!c2ex1 dialog resizeable
  !this.formTitle = |Example Form - Exercise 1|
  !this.initcall  = |!this.init()|
  path down
  frame .inputFrame |Inputs| at x 0 anchor T+L+B
    frame  .tempInputFrame |Temperature conversion (Input)| at x 1
      text .tempInput |Temperature| call |!this.temperatureConvert()| width 10 is REAL
      frame .tempChoiceFrame panel at xmax.tempInput ymin.tempInputFrame + 0.5
        rToggle .celsius |°C| tagwid 2 at xmax.tempInput + 3 ymin.tempInput
        rToggle .fahrenheit |°F| tagwid 2 at xmax.celsius ymin.tempInput
      exit
    exit
    frame .tempRangeFrame |Temperature Range| at x 1 width.tempInputFrame
      text .minimum |Minimim    | call |!this.check(1)| at x 1 width 10 is REAL
      text .maximum |Maximum    | call |!this.check(2)| at x 1 width 10 is REAL
      text .step    |Step Size  | call |!this.check(0)| at x 1 width 10 is REAL format !!INTEGERFMT
      button .fillFahrenheit linklabel |Fill with °F to °C >>|  call |!this.fill('Fahrenheit')| at
xmax.step + 1 ymin.step wid 12
      button .fillCelsius     linklabel |Fill with °C to °F >>|  call |!this.fill('Celsius')| at
xmin.fillFahrenheit ymin.fillFahrenheit - size wid 12
    exit
    frame .stringSplitFrame |Temperature Split| anchor L+B+R at x 1 width.tempInputFrame
      text .stringInput |Input      | width 24 is STRING
      text .delimiter   |Delimiter  | width 10 is STRING
      button .split linklabel |Split temperature >>| call |!this.split()| at xmax.delimiter + 1
ymin.delimiter wid 12
    exit
  exit
  frame .results |Results| at xmax + 1 y 0 anchor ALL
    frame      .tempOutputFrame   |Temperature   conversion   (Output)|   anchor   L+T+R   at   x   1
width.tempInputFrame height.tempInputFrame
      text .tempOutput |Temperature| call || width 10 is REAL
      para .unit at xmax.tempOutput ymin.tempInput text || width 5
    exit
    frame  .tempRangeOutFrame  |Temperature  Results|  anchor  L+T+R  at  x1  width.tempRangeFrame
height.tempRangeFrame
      list .tempList dock fill columns width 1 height 1
    exit
    frame .stringSplitOutFrame |Temperature Split Result| anchor L+B+R at x1 ymin.stringSplitFrame
width.stringSplitFrame height.stringSplitFrame
      text .number |No. of Temp| width 10          is REAL format !!INTEGERFMT
      text .result |Result      | width.stringInput is STRING
    exit
  exit
exit

define method .c2ex1()
  !this.tempChoiceFrame.callback = |!this.temperatureConvert()|
endmethod

define method .init()
  !this.tempInput.val = 0
  !this.tempChoiceFrame.val = 1
  !this.minimum.val = 0
  !this.maximum.val = 100
  !this.step.val = 25
  !this.stringInput.val = |10°C/30°C/20°C/5°C|
  !this.delimiter.val = |/|
  !this.temperatureConvert()
  !this.fill(|Celsius|)
  !this.split()
endmethod
```

Fix1: U not valid

Fix2: REAL with one L

Fix3: Width+height at the end of

Fix4: Incorrect constructor name

A new init method applies default values

```
define method .celsiusToFahrenheit(!celsius is REAL) is REAL
  !fahrenheit = !celsius * 1.8 + 32
  return !fahrenheit
endmethod

define method .fahrenheitToCelsius(!fahrenheit is REAL) is REAL
  !celsius = (!fahrenheit - 32 ) / 1.8
  return !celsius
endmethod

define method .temperatureConvert()
  if !this.tempChoiceFrame.val.eq(1) then
    !this.tempOutput.val = !this.CelsiusToFahrenheit(!this.tempInput.val)
    !this.unit.val = |°F|
  elseif !this.tempChoiceFrame.val.eq(2) then
    !this.tempOutput.val = !this.FahrenheitToCelsius(!this.tempInput.val)
    !this.unit.val = |°C|
  endif
endmethod
```

Fix5: F and C the wrong way around

```
define method .fill(!whichTemperature is STRING)
    !n = 0
    do !temperature from !this.minimum.val to !this.maximum.val by !this.step.val
      !n = !n + 1
      !tempArray[!n][1] = !n.string()
      !tempArray[!n][2] = !temperature.string()
      !tempArray[!n][3] = |=|
      if !whichTemperature.eq(|Celsius|) then
        !fahrenheit = !this.celsiusToFahrenheit(!temperature)
        !tempArray[!n][4] = STRING(!fahrenheit,!!realFMT)
      else
        !celsius = !this.fahrenheitToCelsius(!temperature)
        !tempArray[!n][4] = STRING(!celsius,!!realFMT)
      endif
    enddo
    if !whichTemperature.eq(|Celsius|) then
      !headings = |No. Celsius = Fahrenheit|
    else
      !headings = |No. Fahrenheit = Celsius|
    endif
    !this.tempList.setHeadings(!headings.split())
    !this.tempList.setRows(!tempArray)
endmethod
define method .check(!flag is REAL)
  if !flag.eq(0) then
    if !this.step.val.eq(0) then
      !this.step.val = 1
    endif
  elseif !flag.eq(1).or(!flag.eq(2)) then
    if !this.maximum.val.unset() then
      !this.maximum.val = !this.minimum.val + !this.step.val
    elseif !this.minimum.val.unset() then
      !this.minimum.val = !this.maximum.val - !this.step.val
    endif
    if (!this.minimum.val.lt(-273)) then
      !this.minimum.val = -273
      if (!this.maximum.val.leq(!this.minimum.val)) then
        !this.maximum.val = -272
      endif
    elseif (!this.maximum.val.lt(-273)) then
      !this.minimum.val = -273
      !this.maximum.val = -272
    endif
    if (!this.maximum.val.leq(!this.minimum.val)) then
      if !flag.eq(1) then
        !this.maximum.val = !this.minimum.val + !this.step.val
      elseif !flag.eq(2) then
        !this.minimum.val = !this.maximum.val - !this.step.val
      endif
    endif
  endif
endmethod
```

New method to split the string

```
define method .split()
  !split = !this.stringInput.val.trim().split(!this.delimiter.val)
  !this.number.val = !split.size()
```

```
  !result = ||
  do !n index !split
    if !split[!n].substring(!split[!n].length()).upcase().eq(|C|) then
      !result = !result & !this.celsiusToFahrenheit(!split[!n].substring(1, !split[!n].length() -
2).real()).string(!!INTEGERFMT) & |°F |
    else
      !result = !result & !this.fahrenheitToCelsius(!split[!n].substring(1, !split[!n].length() -
2).real()).string(!!INTEGERFMT) & |°C |
    endif
  enddo
  !this.result.val = !result
endmethod
```

Appendix B2 - Example c2ex2.pmlfrm

```
setup form !!c2ex2 dialog resizeable
  !this.formTitle = |Example Form - Exercise 2|
  !this.initcall  = |!!this.init()|
  path down
  frame .tabset TABSET anchor all
    frame .inputFrame |Inputs| at x 0 y 1
      frame  .tempInputFrame |Temperature conversion (Input)| at x 1 anchor T+L+R
        text .tempInput |Temperature| width 10 is REAL
        frame .tempChoiceFrame panel at xmax.tempInput ymin.tempInputFrame + 0.5
          rToggle .celsius |°C| tagwid 2 at xmax.tempInput + 3 ymin.tempInput
          rToggle .fahrenheit |°F| tagwid 2 at xmax.celsius ymin.tempInput
        exit
      exit
      frame .tempRangeFrame |Temperature Range| at x 1 anchor T+L+R width.tempInputFrame
        text .minimum |Minimim    | call |!!this.check(1)| at x 1 width 10 is REAL
        text .maximum |Maximum    | call |!!this.check(2)| at x 1 width 10 is REAL
        text  .step       |Step  Size    |  call  |!!this.check(0)|  at  x  1  width  10  is  REAL  format
!!INTEGERFMT
      exit
      frame .stringSplitFrame |Temperture Split| anchor all at x 1 width.tempInputFrame
        text .stringInput |Input      | width 24 is STRING
        text .delimiter   |Delimiter  | width 10 is STRING
      exit
    exit
    frame .results |Results| at xmin.inputFrame ymin.inputFrame
      frame   .tempOutputFrame |Temperature conversion (Output)| anchor L+T+R width.tempInputFrame
height.tempInputFrame
        text .tempOutput |Temperature| width 10 is REAL
        para .unit at xmax.tempOutput ymin.tempInput text || width 5
      exit
      frame  .tempRangeOutFrame  |Temperature  Results|  anchor  L+T+R  at  x1  width.tempRangeFrame
height.tempRangeFrame
        list .tempList dock fill columns width 1 height 1
      exit
      frame  .stringSplitOutFrame  |Temperature  Split  Result|  anchor  all  width.stringSplitFrame
height.stringSplitFrame
        text .number |No. of Temp| width 10          is REAL format !!INTEGERFMT
        text .result |Result     | width.stringInput is STRING
      exit
    exit
  exit
  para   .paAcceptChanges at  xmin.tempInputFrame  ymax+0.5 anchor L+B pixmap wid 16 hei 16
  button .lkAcceptChanges |Accept changes| at xmax.paAcceptChanges+1  ymin anchor L+B linklabel wid
10
  button  .lkDiscardChanges  |Discard  changes  |  at  xmax.tempInputFrame  -  size  ymin  anchor  R+B
linklabel wid 10
  para   .paDiscardChanges at  xmin.lkDiscardChanges - 1.5 * size  ymin anchor R+B pixmap wid 16 hei 16
  member .data is ARRAY
  !menu = !this.newMenu(|celsiusMenu|)
  !menu.add(|Callback|, |Switch to °F = °C|, |!this.fill('Fahrenheit')|)
  !menu = !this.newMenu(|fahrenheitMenu|)
  !menu.add(|Callback|, |Switch to °C = °F|, |!this.fill('Celsius')|)
exit

define method .c2ex2()
  !this.results.callback =  |!this.tabCall(|
  !this.paAcceptChanges.addPixmap(!!pml.getPathName('accept.png'))
  !this.paDiscardChanges.addPixmap(!!pml.getPathName('discard.png'))
  !this.lkAcceptChanges.callback = |!this.setData(1)|
  !this.lkDiscardChanges.callback = |!this.setData(2)|
endmethod
```

New menu objects

New Accept/Discard buttons

```
define method .init()
  !this.tempInput.val = 0
  !this.tempChoiceFrame.val = 1
  !this.minimum.val = 0
  !this.maximum.val = 100
  !this.step.val = 25
  !this.stringInput.val = |10°C/30°C/20°C/5°C|
  !this.delimiter.val = |/|
  !this.setData(1)
endmethod

define method .tabCall(!gad is GADGET, !type is STRING)
  if !type eq |SHOWN| then
    !this.temperatureConvert()
    !this.fill(|Celsius|)
    !this.split()
  endif
endmethod

define method .setData(!flag is REAL)
  -- Either save the values, or bring them back (array of strings)
  if !flag.eq(1) then
    !this.data[1] = !this.tempInput.val
    !this.data[2] = !this.tempChoiceFrame.val
    !this.data[3] = !this.minimum.val
    !this.data[4] = !this.maximum.val
    !this.data[5] = !this.step.val
    !this.data[6] = !this.stringInput.val
    !this.data[7] = !this.delimiter.val
  else
    if !this.data.size().eq(7) then
      !this.tempInput.val       = !this.data[1]
      !this.tempChoiceFrame.val = !this.data[2]
      !this.minimum.val         = !this.data[3]
      !this.maximum.val         = !this.data[4]
      !this.step.val            = !this.data[5]
      !this.stringInput.val     = !this.data[6]
      !this.delimiter.val       = !this.data[7]
      !this.temperatureConvert()
      !this.fill(|Celsius|)
      !this.split()
    endif
  endif
endmethod
```

Opencallback on the Results tab

Method to transfer to and from the .data member

Appendix B3 - Example c2ex3.pmlfrm

```
setup form !!c2ex3
  path down
  !this.formTitle = |PML Objects - Exercise 3|
  textpane .txtp 'Input Text:' at x 0 ymax wid 40 hei 10
  button .save |Save| at xmax form - size wid 6
  member .file is FILE
exit

define method .c2ex3()
  !this.file.file('C:\temp\c2ex3text.txt')
  !this.save.callback = |!!fileBrowser(!this.file.pathname(), !this.file.entry(), $
    'Save File', FALSE, '!!c2ex3.save(!!fileBrowser.file)')|
endmethod

define method .save(!file is file)
  !this.file = !file
  !this.file.writefile('OVERWRITE', !this.txtp.val)
endmethod
```

Appendix B4 - Example cuboid.pmlobj

```
define object CUBOID
  member .xlen is REAL
  member .ylen is REAL
  member .zlen is REAL
endobject

define method .cuboid()
  !this.xlen = 0
  !this.ylen = 0
  !this.zlen = 0
endmethod

define method .cuboid(!x is REAL, !y is REAL, !z is REAL)
  !this.xlen = !x
  !this.ylen = !y
  !this.zlen = !z
endmethod

define method .volume() is REAL
  return !this.xlen * !this.ylen * !this.zlen
endmethod

define method .surfaceArea() is REAL
  return !this.xlen * !this.zlen * 2 + !this.ylen * !this.zlen * 2 + !this.xlen * !this.ylen * 2
endmethod
```

Appendix B5 - Example datastore.pmlobj

```
define object DATASTORE
  member .formTextGadgets is ARRAY
  member .gadgetValues is ARRAY
endobject

define method .dataStore(!form is form)
  !this.formTextGadgets.clear()
  !this.gadgetValues.clear()
  !formMembers = !form.attributes()
  do !n index !formMembers
    if !form.attribute(!formMembers[!n]).objectType().eq(|GADGET|) then
      if !form.attribute(!formMembers[!n]).type().eq(|TEXT|) then
        !this.formTextGadgets.append(!form.attribute(!formMembers[!n]))
        !this.gadgetValues.append(!form.attribute(!formMembers[!n]).val)
      endif
    endif
  enddo
endmethod

define method .updateInfo()
  do !n index !this.formTextGadgets
    !this.gadgetValues[!n] = !this.formTextGadgets[!n].val
  enddo
endmethod

define method .restoreInfo()
  do !n index !this.formTextGadgets
    !this.formTextGadgets[!n].val = !this.gadgetValues[!n]
  enddo
endmethod
```

Appendix B6 - Example c2ex4.pmlfrm

```
setup form !!c2ex4
  !this.formTitle     = |Equipment Checker|
  !this.initcall      = |!this.init()|
  !this.newMenu(|pop1|)
  !this.pop1.add('CALLBACK', 'Go to Nozzle', '!!CE = !this.nozz.selection().dbref()', 'NOZZ')
  para     .title text |Available Equipment|
  line     .titleLine at xmin.title ymax.title horiz wid 48
  combo    .equip at xmin.title + 0.5 ymax.titleLine + 0.2 call |!this.validate(| width 10
  list     .nozz  at xmin.equip ymax.equip width 45 length 5
  button   .site linklabel |Update| at xmax.equip + 0.5 ymin.equip call |!this.init()|
  button       .refreshNozz   at   xmax.nozz   -   size   ymax.nozz   tooltip   |Refresh   Nozzles|   call
|!this.checkNozz()| pixmap wid 16 hei 16
  textpane .atta |Equipment Attributes| at xmin.equip ymax.refreshNozz width 47 height 3.5
```

```
  button      .updateAtta  linklabel  |Update  Atts|  at  xmax.nozz  -  size  ymax.atta  +  0.2  call
|!this.updateAtt(2)| wid 7
exit

define method .c2ex4()
  !nozz = |Name/Connected?/Attached?/Aligned?/Size check?|
  !this.nozz.setHeadings(!nozz.split(|/|))
  !info[1] = 'Description - Unset'
  !info[2] = 'Function - Unset'
  !info[3] = 'Purpose - Unset'
  !this.atta.val = !info
  !this.nozz.setpopup(!this.pop1)
  !this.refreshNozz.addPixmap(!!pml.getPathName(|refresh16.png|))
endmethod

define method .init()
  if !!CE.type.eq(|SITE|) then
    !level = |SITE|
  else
    !level = |ZONE|
  endif
  VAR !coll COLL ALL EQUIP FOR $!level
  handle ANY
    !!Alert.Error(|Make sure you are at an EQUI, ZONE or SITE element|)
  elsehandle NONE
    if !coll.size().eq(0) and !level.eq(|ZONE|) then
      VAR !coll COLL ALL EQUIP FOR SITE
      !!alert.Message(|No equipment in current ZONE, so whole SITE will be searched|)
    endif
    VAR !name EVAL FLNN FOR ALL FROM !coll
    !this.equip.dtext = !name
    !this.equip.rtext = !coll
    !this.collNozz()
  endhandle
endmethod

define method .validate(!gad is GADGET, !event is STRING)
  if !event.eq('VALIDATE') then
    !userInput = !this.equip.displayText()
    do !n index !this.equip.dtext
      !Chrs = !userInput.length()
      !test = !this.equip.dtext[!n].upcase().substring(1, !Chrs)
      if !userInput.upcase().eq(!test) then
        !this.equip.val = !n
        break
      endif
    enddo
  endif
  !this.collNozz()
endmethod

define method .collNozz()
  !equipRef = !this.equip.selection().dbref()
  if !equipRef.unset() then
    !this.nozz.clear()
  else
    !nozzColl = object COLLECTION()
    !nozzColl.type('NOZZ')
    !nozzColl.scope(!equipRef)
    !results = !nozzColl.results()
    !this.nozz.dtext = !results.evaluate(object block ('!results[!evalIndex].flnn'))
    !this.nozz.rtext = !results.evaluate(object block ('!results[!evalIndex].string()'))
    !this.checkNozz()
    !this.updateAtt(1)
  endif
endmethod

define method .checkNozz()
  if !this.nozz.rtext.unset().not() then
    do !n index !this.nozz.rtext
      !nozz = !this.nozz.rtext[!n].dbref()
      do !m from 2 to 4
        !result[!m] = |N/A|
      enddo
      if !nozz.cref.unset().or(!nozz.cref.badref()) then
        !result[1] = |Check|
      else
```

```
        !result[1] = |OK|
        !end = |t|
        if !nozz.cref.href.eq(!nozz) then
          !end = |h|
        endif
        if !nozz.pos.wrt( /* ).eq(!nozz.cref.attribute(!end & |pos|).wrt( /* )) then
          !result[2] = |OK|
        endif
        if !nozz.pdir[1].wrt( /* ).eq(!nozz.cref.attribute(!end & |dir|).wrt( /* )) then
          !result[3] = |OK|
        endif
        if !nozz.cpar[1].eq(!nozz.cref.attribute(!end & |bore|).real()) then
          !result[4] = |OK|
        endif
      endif
      !nozzInfo[!n][1] = !nozz.flnn
      !nozzInfo[!n].appendArray(!result)
    enddo
    !rtext = !this.nozz.rtext
    !this.nozz.setrows(!nozzInfo)
    !this.nozz.rtext = !rtext
  endif
endmethod

define method .updateAtt(!flag is REAL)
  !equip = !this.equip.selection().dbref()
  !availAtts = !equip.attributes()
  !this.atta.tag = !equip.flnn & ' Attributes'
  !rows = !this.atta.val
  !out = ARRAY()
  do !n index !rows
    !text = !rows[!n]
    !split = !text.split('-')
    !attrib = !split[1].trim()
    !avail   =   !availAtts.evaluate(object   block   ('!availAtts[!evalIndex].upcase().substring(1,
!attrib.length())'))
    if !avail.findfirst(!attrib.upcase()).unset().not() then
      if !flag.eq(1) then
        !out.append(!attrib & | - | & !equip.attribute(!availAtts[!avail.findfirst(!attrib.upcase())]))
      elseif !flag.eq(2) then
        !val = !split[2].trim()
        !equip.attribute(!availAtts[!avail.findfirst(!attrib.upcase())]).assign(!val)
        !out.append(!attrib & | - | & !val)
      endif
    endif
  enddo
  !this.atta.val = !out
endmethod
```

Appendix B7 - Example exampleVolumeView.pmlfrm

```
setup form !!exampleVolumeView
  !this.formTitle = |Volume View|
  !this.initcall  = |!this.init()|
  !this.firstShownCall  = |!this.firstShown()|
  !this.killingCall     = |!this.close()|
  !this.newMenu(|pop1|)
  !this.pop1.add('CALLBACK', 'Limits CE', '!this.limitsCE()')
  !pix = |pixmap wid 16 hei 16|
  path DOWN
  para .prompt text |Navigate : | width 46 lines 1
  button .but1 tooltip |Limits CE| call |!this.limitsCE()| $!pix
  button .but2 tooltip |Walk to Drawlist| call |!this.walkDrawlist()| $!pix
  button .but3 tooltip |Add to Drawlist| call |!this.setDrawlist(1, FALSE)| $!pix
  button .but4 tooltip |Remove from Drawlist| call |!this.setDrawlist(2, FALSE)| $!pix
  view .volumeView at xmax.but1 ymax.prompt volume
    width 46 height 15
    limits auto
    isometric 3
  exit
  member .drawlist is REAL
exit

define method .exampleVolumeView()
  !this.but1.addPixmap(!!pml.getPathName(|autoce16.png|))
  !this.but2.addPixmap(!!pml.getPathName(|ng_zoomtodrawlist.png|))
  !this.but3.addPixmap(!!pml.getPathName(|addce16.png|))
```

```
  !this.but4.addPixmap(!!pml.getPathName(|removece16.png|))
  !this.volumeView.borders = FALSE
  !this.volumeView.background = |darkslate|
  !this.volumeView.shaded = TRUE
  !this.volumeView.projection = |PARALLEL|
  !this.volumeView.radius = 100
  !this.volumeView.range = 500.0
  !this.volumeView.eyemode = FALSE
  !this.volumeView.step = 25
  !this.volumeView.setpopup(!this.pop1)
  -- Create local drawlist within the global object
  !this.drawlist = !!gphDrawlists.createDrawList()
endmethod

define method .firstShown()
  -- Add 3D view to view system
  !!gphViews.add(!this.volumeView)
  -- Add local drawlist add to 3D view
  !!gphDrawlists.attachView(!this.drawlist, !this.volumeView)
  --!!gphDrawlists.drawlists[!this.drawlist].holes(TRUE)
endmethod

define method .init()
  !this.setDrawlist(1, TRUE)
  -- Active the volume view
  !this.volumeView.active = TRUE
endmethod

define method .close()
  !!gphDrawlists.detachView(!this.volumeView)
  !!gphDrawlists.deleteDrawlist(!this.drawlist)
endmethod

define method .walkDrawlist()
  !drawlist = !!gphDrawlists.drawlist(!this.drawlist)
  -- derive a volume object from the members of the drawlist
  !volume = object volume(!drawlist.members())
  !limits[1]    = !volume.from.east
  !limits[2]    = !volume.to.east
  !limits[3]    = !volume.from.north
  !limits[4]    = !volume.to.north
  !limits[5]    = !volume.from.up
  !limits[6]    = !volume.to.up
  !this.volumeView.limits = !limits
endmethod

define method .limitsCE()
  -- Set the Views limits based on the chosen element
  !!gphViews.limits(!this.volumeView, !!CE)
endmethod

define method .setDrawlist(!flag is REAL, !reset is BOOLEAN)
  !drawlist = !!gphDrawlists.drawlist(!this.drawlist)
  -- Clear the drawlist is a reset is requested
  if !reset then
    !drawlist.removeall()
  endif
  -- Add/Remove CE
  if !flag.eq(1) then
    !drawlist.add(!!CE)
  elseif !flag.eq(2) then
    !drawlist.remove(!!CE)
  endif
  !this.walkDrawlist()
endmethod
```

Appendix B8 - Example c2ex5.pmlfrm

```
setup form !!c2ex5
  !this.formTitle     = |Equipment Checker|
  !this.initcall      = |!this.init()|
  !this.firstShownCall  = |!this.firstShown()|
  !this.killingCall     = |!this.close()|
  !this.quitcall      = |!this.clear()|
  !this.newMenu(|pop1|)
  !this.pop1.add('CALLBACK', 'Go to Nozzle', '!!CE = !this.nozz.selection().dbref()', 'NOZZ')
  para .prompt text |Navigate : | width 46 lines 1
  frame .view panel at x 0.5 ymax.prompt wid 1 hei 1
    view .volumeView at x 0.5 ymax.prompt VOLUME
      width 50 aspect 0.707
      border off
      shading on
      isometric 3
    exit
  exit
  para     .title at xmax.view + 0.5 ymin.prompt text |Available Equipment|
  line     .titleLine at xmin.title ymax.title horiz wid 48
  combo    .equip at xmin.title + 0.5 ymax.titleLine + 0.2 call |!this.validate(| width 10
  list     .nozz  at xmin.equip ymax.equip call |!this.check()| width 45 length 5
  button   .site linklabel |Update| at xmax.equip + 0.5 ymin.equip call |!this.init()|
  button   .refreshNozz at xmax.nozz - size ymax.nozz tooltip |Refresh Nozzles| call |!this.checkNozz()|
pixmap wid 16 hei 16

  textpane .atta |Equipment Attributes| at xmin.equip ymax.refreshNozz width 47 height 3.5
  button       .updateAtta  linklabel  |Update  Atts|  at  xmax.nozz  -  size  ymax.atta  +  0.2  call
|!this.updateAtt(2)| wid 7

  para     .taskTitle at xmin.title ymax.atta + 0.5 text |Available Tasks|
  line     .taskLine at xmin.title ymax.taskTitle horiz wid.titleLine

  para     .clipPix at xmin.equip + 1 ymax.taskLine + 0.2 pixmap wid 16 hei 16
  button   .clipBut    linklabel at xmax.clipPix + 1 ymin.clipPix |Add Connected and Enable Clipbox...|
call |!this.enableClip()|
  para     .tagNozzPix at xmin.equip + 1 ymax.clipPix + 0.2 pixmap wid 16 hei 16
  button     .tagNozzBut  linklabel  at  xmax.clipPix  +  1  ymin.tagNozzPix  |Tag  Selected  Nozzle|  call
|!this.tag(!this.tagNozzBut)|
  para     .tagNozzlesPix at xmin.equip + 1 ymax.tagNozzPix+ 0.2 pixmap wid 16 hei 16
  button     .tagNozzlesBut  linklabel  at  xmax.clipPix  +  1  ymin.tagNozzlesPix  |Tag  All  Nozzles|  call
|!this.tag(!this.tagNozzlesBut)|
  para     .hlNozzPix at xmin.equip + 1 ymax.tagNozzlesPix+ 0.2 pixmap wid 16 hei 16
  button       .hlNozzBut  linklabel  at  xmax.clipPix  +  1  ymin.hlNozzPix  |Highlight  Nozzles|  call
|!this.highlight()|
  frame .limitslide foldup |Edit Clip Volume| at xmin.atta ymax.hlNozzPix + 0.5 width 46
    slider .slide call |!this.slide()| dock t horizontal range 0 +2000 step 100 val 0 width 1 height 1.5
    text    .minslide at xmin.limitslide+0.2  ymin.limitslide+2.5 call |!this.updateRange()| width 5 is
REAL
    text    .maxslide at xmax.limitslide-size ymin.limitslide+2.5 call |!this.updateRange()| width 5 is
REAL
    toggle .incl |Update view limits?| at xmax.minslide + 5 ymin.limitslide+2.5
  exit
  frame .colour foldup |Edit Highlight Colour| at xmin.atta ymax.limitslide width.limitslide
    para .para1       at xmin.colour + 0.5 ymin.colour + 1.2 text |Current equipment  |
    button .butt1 | | at xmax.para1 - 3 ymin.para1 call |!this.colourpick('A' )| pixmap wid 30 hei 15
    para .para2       at xmin.colour + 0.5 ymax.butt1        text |Connected nozzles  |
    button .butt2 |  | at xmax.para2 - 3 ymax.butt1 call |!this.colourpick('B' )| pixmap wid 30 hei 15
    para .para3       at xmin.colour + 0.5 ymax.butt2        text |Unconnected nozzles|
    button .butt3 |  | at xmax.para3 - 3 ymax.butt2 call |!this.colourpick('C' )| pixmap wid 30 hei 15
    para .para4       at xmin.colour + 0.5 ymax.butt3        text |Check nozzles      |
    button .butt4 |  | at xmax.para3 - 3 ymax.butt3 call |!this.colourpick('D' )| pixmap wid 30 hei 15

    frame .colourpick panel at xmax.butt1  + 3.5 ymin.colour + 0.5 width 10
      !val = 0
      do !I from 1 to 10
        do !J from 0 to 4
          !val = !val + 1
          !no = !I & |000| & !J
          !x = (!I * 2) - 1
          !y = (!J * 0.6) + 0.75
          button .butt$!no | | pixmap at x$!x ymin.colourpick + $!y call |!this.colourpick($!val)|
backg $!val width 10 aspect 1
        enddo
      enddo
    exit
```

```
  exit
  member .clipbox is GPHCLIPBOX
  member .drawlist is REAL
  member .limits is ARRAY
  member .col    is REAL
  member .tagged is ARRAY
  member .highlight is ARRAY
exit

define method .c2ex5()
  !nozz = |Name/Connected?/Attached?/Aligned?/Size check?|
  !this.nozz.setHeadings(!nozz.split(|/|))
  !info[1] = 'Description - Unset'
  !info[2] = 'Function - Unset'
  !info[3] = 'Purpose - Unset'
  !this.atta.val = !info
  !this.nozz.setpopup(!this.pop1)
  !this.volumeView.borders = FALSE
  !this.volumeView.background = |darkslate|
  !this.volumeView.shaded = TRUE
  !this.volumeView.projection = |PARALLEL|
  !this.volumeView.radius = 100
  !this.volumeView.range = 500.0
  !this.volumeView.eyemode = FALSE
  !this.volumeView.step = 25
  !this.minslide.val = 0
  !this.maxslide.val = 2000
  !this.clipPix.addPixmap(!!pml.getPathName(|clip_off.png|))
  !this.refreshNozz.addPixmap(!!pml.getPathName(|refresh16.png|))
  !this.tagNozzPix.AddPixmap(!!PML.GetPathName('single.png'))
  !this.tagNozzlesPix.AddPixmap(!!PML.GetPathName('multi.png'))
  !this.hlNozzPix.AddPixmap(!!PML.GetPathName('highlight.png'))
  !this.colourpick.visible = FALSE
  !this.limitslide.visible  = FALSE
  !this.limitslide.expanded = FALSE
  !this.colour.visible  = FALSE
  !this.colour.expanded = FALSE
  !this.drawlist = !!gphDrawlists.createDrawList()
  !this.clipbox = object GPHCLIPBOX()
  !this.clipbox.view = !this.volumeView
  !this.clipbox.capOn()
  !this.butt1.background = 1
  !this.butt2.background = 20
  !this.butt3.background = 18
  !this.butt4.background = 23
endmethod

define method .firstShown()
  -- Add 3D view to view system
  !!gphViews.add(!this.volumeView)
  -- Add local drawlist add to 3D view
  !!gphDrawlists.attachView(!this.drawlist, !this.volumeView)
endmethod

define method .close()
  !!gphDrawlists.detachView(!this.volumeView)
  !!gphDrawlists.deleteDrawlist(!this.drawlist)
endmethod

define method .init()
  !this.volumeView.active = TRUE
  if !!CE.type.eq(|SITE|) then
    !level = |SITE|
  else
    !level = |ZONE|
  endif
  VAR !coll COLL ALL EQUIP FOR $!level
  handle ANY
    !!Alert.Error(|Make sure you are at an EQUI, ZONE or SITE element|)
  elsehandle NONE
    if !coll.size().eq(0) and !level.eq(|ZONE|) then
      VAR !coll COLL ALL EQUIP FOR SITE
      !!alert.Message(|No equipment found in current ZONE, so whole SITE will be searched|)
    endif
    VAR !name EVAL FLNN FOR ALL FROM !coll
    !this.equip.dtext = !name
    !this.equip.rtext = !coll
```

```
    !this.collNozz()
  endhandle
endmethod

define method .validate(!gad is GADGET, !event is STRING)
  if !event.eq('VALIDATE') then
    !userInput = !this.equip.displayText()
    do !n index !this.equip.dtext
      !Chrs = !userInput.length()
      !test = !this.equip.dtext[!n].upcase().substring(1, !Chrs)
      if !userInput.upcase().eq(!test) then
        !this.equip.val = !n
        break
      endif
    enddo
  endif
  !this.collNozz()
endmethod

define method .collNozz()
  !equipRef = !this.equip.selection().dbref()
  if !equipRef.unset() then
    !this.nozz.clear()
  else
    !nozzColl = object COLLECTION()
    !nozzColl.type('NOZZ')
    !nozzColl.scope(!equipRef)
    !results = !nozzColl.results()
    !this.nozz.dtext = !results.evaluate(object block ('!results[!evalIndex].flnn'))
    !this.nozz.rtext = !results.evaluate(object block ('!results[!evalIndex].string()'))
    !this.checkNozz()
    !this.clear()
    !this.updateAtt(1)
    !this.setDrawlist()
    !this.enableClip()
    !this.highlight()
    !this.tagged.clear()
    do !n index !this.nozz.dtext
      !this.tagged[!n] = FALSE
    enddo
  endif
endmethod

define method .checkNozz()
  if !this.nozz.rtext.unset().not() then
    do !n index !this.nozz.rtext
      !nozz = !this.nozz.rtext[!n].dbref()
      do !m from 2 to 4
        !result[!m] = |N/A|
      enddo
      if !nozz.cref.unset().or(!nozz.cref.badref()) then
        !result[1] = |Check|
      else
        !result[1] = |OK|
        !end = |t|
        if !nozz.cref.href.eq(!nozz) then
          !end = |h|
        endif
        if !nozz.pos.wrt( /* ).eq(!nozz.cref.attribute(!end & |pos|).wrt( /* )) then
          !result[2] = |OK|
        endif
        if !nozz.pdir[1].wrt( /* ).eq(!nozz.cref.attribute(!end & |dir|).wrt( /* )) then
          !result[3] = |OK|
        endif
        if !nozz.cpar[1].eq(!nozz.cref.attribute(!end & |bore|).real()) then
          !result[4] = |OK|
        endif
      endif
      !nozzInfo[!n][1] = !nozz.flnn
      !nozzInfo[!n].appendArray(!result)
    enddo
    !this.highlight = !nozzInfo
    !rtext = !this.nozz.rtext
    !this.nozz.setrows(!nozzInfo)
    !this.nozz.rtext = !rtext
  endif
endmethod
```

```
define method .updateAtt(!flag is REAL)
  !equip = !this.equip.selection().dbref()
  !availAtts = !equip.attributes()
  !this.atta.tag = !equip.flnn & ' Attributes'
  !rows = !this.atta.val
  !out = ARRAY()
  do !n index !rows
    !text = !rows[!n]
    !split = !text.split('-')
    !attrib = !split[1].trim()
    !avail   =   !availAtts.evaluate(object   block   ('!availAtts[!evalIndex].upcase().substring(1,
!attrib.length())'))
    if !avail.findfirst(!attrib.upcase()).unset().not() then
      if !flag.eq(1) then
        !out.append(!attrib & | - | &
!equip.attribute(!availAtts[!avail.findfirst(!attrib.upcase())]))
      elseif !flag.eq(2) then
        !val = !split[2].trim()
        !equip.attribute(!availAtts[!avail.findfirst(!attrib.upcase())]).assign(!val)
        !out.append(!attrib & | - | & !val)
      endif
    endif
  enddo
  !this.atta.val = !out
endmethod

define method .setDrawlist()
  !equip = !this.equip.selection().dbref()
  !drawlist = !!gphDrawlists.drawlist(!this.drawlist)
  !drawlist.removeall()
  !drawlist.add(!equip)
  !!gphViews.limits(!this.volumeView, !equip)
endmethod

define method .slide()
  !value = !this.slide.val
  if !this.incl.val.eq(TRUE) then
    !limits = !this.limits
    !modlimit[1] = !limits[1] - !value
    !modlimit[2] = !limits[2] + !value
    !modlimit[3] = !limits[3] - !value
    !modlimit[4] = !limits[4] + !value
    !modlimit[5] = !limits[5] - !value
    !modlimit[6] = !limits[6] + !value
    !this.volumeView.limits = !modlimit
  endif
  !this.volumeView.clipBoxXlen = !this.clipbox.box.xlength + !value
  !this.volumeView.clipBoxYlen = !this.clipbox.box.ylength + !value
  !this.volumeView.clipBoxZlen = !this.clipbox.box.zlength + !value
  !this.volumeView.refresh()
endmethod

define method .updateRange()
  !range[1] = !this.minslide.val
  !range[2] = !this.maxslide.val
  !range[3] = 100
  !this.slide.range = !range
  !this.slide.refresh()
endmethod

define method .enableClip()
  !drawlist = !!gphDrawlists.drawlist(!this.drawlist)
  !connectNozz = ARRAY()
  do !n index !this.nozz.rtext
    !nozz = !this.nozz.rtext[!n]
    if !nozz.dbref().cref.unset().not().and(!nozz.dbref().cref.badref().not()) then
      !connectNozz.append(!nozz.dbref().cref)
    endif
  enddo

  !volume = object volume(!this.equip.selection().dbref())
  !this.clipbox.box = !volume.box()
  if !this.clipBut.val then
    !this.clipBut.tag = |Remove Connected and Disable Clipbox...|
    !this.clipPix.addPixmap(!!pml.getPathName(|clip_on.png|))
    !this.limitslide.visible = TRUE
    do !n index !connectNozz
```

```
      !drawlist.add(!connectNozz[!n])
    enddo
    !this.limits = !this.volumeView.limits
    !this.clipbox.active = FALSE
    !this.clipbox.set()
    !this.clipbox.active = TRUE
    !this.volumeView.clipping = TRUE
    !this.slide()
  else
    !this.clipBut.tag = |Add Connected and Enable Clipbox...|
    !this.clipPix.addPixmap(!!pml.getPathName(|clip_off.png|))
    !this.limitslide.visible = FALSE
    !this.limitslide.expanded = FALSE
    do !n index !connectNozz
      !drawlist.remove(!connectNozz[!n])
    enddo
    !this.clipbox.active = FALSE
    !this.volumeView.clipping = FALSE
  endif
endmethod

define method .tag(!gad is GADGET)
  if  !gad.tag.upcase().eq(|TAG  SELECTED  NOZZLE|).or(!gad.tag.upcase().eq(|UNTAG  SELECTED  NOZZLE|))
then
    !start = !this.nozz.val
    !finish = !this.nozz.val
  else
    !start = 1
    !finish = !this.nozz.dtext.size()
  endif

  if !gad.val.eq(TRUE) then
    do !n from !start to !finish
      if !this.tagged[!n].eq(FALSE) then
        !nozzname = !this.nozz.rtext[!n].dbref().flnn
        !nozzpos  = !this.nozz.rtext[!n].dbref().pos
        !num = 1500 + !n
        AID TEXT NUMBER $!num '$!nozzname' AT $!nozzpos
        !this.tagged[!n] = TRUE
      endif
    enddo
  else
    do !n from !start to !finish
      !num = 1500 + !n
      AID CLEAR text $!num
      handle ANY
      endhandle
      !this.tagged[!n] = FALSE
    enddo
  endif
  !testTagged = !this.tagged
  !testTagged.unique()
  if !testtagged.size().eq(1).and(!testtagged[1].eq(TRUE)) then
    !this.tagNozzlesBut.val = TRUE
    !this.tagNozzlesBut.tag = |Untag All Nozzles|
    !this.tagNozzlesPix.addPixmap(!!pml.getPathName(|multi_x.png|))
  else
    !this.tagNozzlesBut.val = FALSE
    !this.tagNozzlesBut.tag = |Tag All Nozzles|
    !this.tagNozzlesPix.addPixmap(!!pml.getPathName(|multi.png|))
  endif
  !this.check()
endmethod

define method .check()
  !check = !this.tagged[!this.nozz.val]
  if !check.eq(TRUE) then
    !this.tagNozzBut.val = TRUE
    !this.tagNozzBut.tag = |Untag Selected Nozzle|
    !this.tagNozzPix.addPixmap(!!pml.getPathName(|single_x.png|))
  else
    !this.tagNozzBut.val = FALSE
    !this.tagNozzBut.tag = |Tag Selected Nozzle|
    !this.tagNozzPix.addPixmap(!!pml.getPathName(|single.png|))
  endif
endmethod
```

```
define method .colourpick(!colour is ANY)
  !col = !colour.string()
  if !col.eq('A') then
    !this.col = 1
    !this.colourpick.visible = TRUE
  elseif !col.eq('B') then
    !this.col = 2
    !this.colourpick.visible = TRUE
  elseif !col.eq('C') then
    !this.col = 3
    !this.colourpick.visible = TRUE
  elseif !col.eq('D') then
    !this.col = 4
    !this.colourpick.visible = TRUE
  else
    if !this.col.eq(1) then
      !this.butt1.background = !col.real()
    elseif !this.col.eq(2) then
      !this.butt2.background = !col.real()
    elseif !this.col.eq(3) then
      !this.butt3.background = !col.real()
    elseif !this.col.eq(4) then
     !this.butt4.background = !col.real()
    endif
    !this.colourpick.visible = FALSE
    !this.highlight()
  endif
endmethod

define method .highlight()
  !equip = !this.equip.selection().dbref()
  if !this.hlNozzBut.val.eq(TRUE) then
    !!gphDrawlists.drawlists[!this.drawlist].highlight(!equip,!this.butt1.background)
    !this.hlNozzPix.AddPixmap(!!PML.GetPathName('highlight_x.png'))
    !this.hlNozzBut.tag = |Unhightlight Nozzles|
    !this.colour.visible = TRUE
    if !this.nozz.rtext.unset().not() then
      !nozz = !this.highlight
      do !I index !nozz
        !nozzle = !this.nozz.rtext[!I].dbref()
        if !nozz[!I][2].eq(|OK|) then
          !col = !this.butt2.background
        elseif !nozz[!I][2].eq(|Check|) then
          !col = !this.butt3.background
        endif
        if !nozz[!I][3].eq(|Check|) or !nozz[!I][4].eq(|Check|) or !nozz[!I][5].eq(|Check|) then
          !col = !this.butt4.background
        endif
        !!gphDrawlists.drawlists[!this.drawlist].highlight(!nozzle,!col)
      enddo
    endif
  else
    !this.hlNozzPix.AddPixmap(!!PML.GetPathName('highlight.png'))
    !this.hlNozzBut.tag = |Hightlight Nozzles|
    !this.colour.visible = FALSE
    !this.colour.expanded = FALSE
    !!gphDrawlists.drawlists[!this.drawlist].unhighlight(!equip)
    if !this.nozz.rtext.unset().not() then
      do !i index !this.nozz.rtext
        !!gphDrawlists.drawlists[!this.drawlist].unhighlight(!this.nozz.rtext[!i].dbref())
      enddo
    endif
  endif
endmethod

define method .clear()
  !equip = !this.equip.selection().dbref()
  !!gphDrawlists.drawlists[!this.drawlist].unhighlight(!equip)
  do !n index !this.nozz.rtext
    !num = 1500 + !n
    AID CLEAR text $!num
    handle ANY
    endhandle
  enddo
  !this.tagNozzBut.val = FALSE
  !this.tagNozzBut.tag = |Tag Selected Nozzle|
  !this.tagNozzPix.addPixmap(!!pml.getPathName(|single.png|))
```

```
  !this.tagNozzlesBut.val = FALSE
  !this.tagNozzlesBut.tag = |Tag All Nozzles|
  !this.tagNozzlesPix.addPixmap(!!pml.getPathName(|multi.png|))
endmethod
```

Appendix B9 - Example c2ex6.pmlfrm – part 1

```
setup form !!c2ex6
  !this.formTitle     = |Equipment Checker|
  !this.initcall      = |!this.init()|
  !this.newMenu(|pop1|)
  !this.pop1.add('CALLBACK', 'Go to Nozzle', '!!CE = !this.nozz.selection().dbref()')
  para     .title text |Available Equipment|
  line     .titleLine at xmin.title ymax.title horiz wid 48
  button   .pick  tooltip 'Identify Equipment' pixmap at xmin.title + 0.5 ymax.titleLine + 0.2 call
|!this.pick()| width 16 height 16
  option .equip at xmax.pick ymin.pick  call |!this.collNozz()| width 10
  list     .nozz  at xmin.pick ymax.equip width 45 length 5
  button   .site linklabel |Update| at xmax.equip + 0.5 ymin.equip call |!this.init()|
exit

define method .c2ex6()
  !this.nozz.setpopup(!this.pop1)
  !this.pick.AddPixmap(!!PML.GetPathName('pickidentify.png'))
endmethod

define method .pick()
  !packet = object EDGPACKET()
  !packet.elementPick(|Identify Equipment|)
  !packet.description = |Identify Equipment|
  !packet.action = |!!c2ex6.identify(!this.return)|
  !!edgCntrl.add(!packet)
endmethod

define method .identify(!pickInfo is ANY)
  !item = !pickInfo[1].item
  !!edgCntrl.remove(|Identify Equipment|)
  -- Loop to find the EQUI element
  do
    break if (!item.type eq |EQUI|) or (!item.type eq |WORL|)
    !item = !item.owner
  enddo
  if !item.type eq |WORL| then
    !!Alert.Warning(|Choose a piece of equipment|)
    !this.pick()
  else
    -- Find the EQUI in the OPTION gadget
    !this.equip.val = !this.equip.dtext.FindFirst(!item.flnn)
    handle ANY
      !!Alert.Warning(|The picked equipment is not avaiable in the current OPTION gadget|)
    endhandle
    !this.collNozz()
  endif
endmethod

define method .init()
  if !!CE.type.eq(|SITE|) then
    !level = |SITE|
  else
    !level = |ZONE|
  endif
  VAR !coll COLL ALL EQUIP FOR $!level
  handle ANY
    !!Alert.Error(|Make sure you are at an EQUI, ZONE or SITE element|)
  elsehandle NONE
    if !coll.size().eq(0) and !level.eq(|ZONE|) then
      VAR !coll COLL ALL EQUIP FOR SITE
      !!alert.Message(|No equipment found in current ZONE, so whole SITE will be searched|)
    endif
    VAR !name EVAL FLNN FOR ALL FROM !coll
    !this.equip.dtext = !name
    !this.equip.rtext = !coll
    !this.collNozz()
  endhandle
endmethod
```

```
define method .collNozz()
  !equipRef = !this.equip.selection().dbref()
  if !equipRef.unset() then
    !this.nozz.clear()
  else
    !nozzColl = object COLLECTION()
    !nozzColl.type('NOZZ')
    !nozzColl.scope(!equipRef)
    !results = !nozzColl.results()
    !this.nozz.dtext = !results.evaluate(object block ('!results[!evalIndex].flnn'))
    !this.nozz.rtext = !results.evaluate(object block ('!results[!evalIndex].string()'))
  endif
endmethod
```

Appendix B10 - Example c2ex6.pmlfrm – part 2

```
  ------------------------------------------
  --
  --  Modifications made to these methods:
  --
  ------------------------------------------

define method .pick()
  !packet = object EDGPACKET()
  !packet.elementPick(|Identify EQUI or NOZZ - Escape to cancel|)
  !packet.description = |Identify Element|
  !packet.action = |!!c2ex6.identify(!this.return)|
  !!edgCntrl.add(!packet)
endmethod

define method .identify(!pickInfo is ANY)
  !item = !pickInfo[1].item
  !!edgCntrl.remove(|Identify Element|)
  -- Loop to find the EQUI element
  if !item.type.neq(|NOZZ|) then
    do
      break if (!item.type.eq(|EQUI|).or(!item.type.eq(|WORL|)))
      !item = !item.owner
    enddo
  endif
  if !item.type.eq(|WORL|) then
    !!Alert.Warning(|Choose either a nozzle or piece of equipment|)
    !this.pick()
  elseif !item.type.eq(|EQUI|) then
    -- Find the EQUI in the OPTION gadget
    !this.equip.val = !this.equip.rtext.findFirst(!item.string())
    !this.collNozz()
  elseif !item.type.eq(|NOZZ|) then
    -- Pick the chosen NOZZ
    if !this.nozz.rtext.findFirst(!item.string()).unset() then
      !equi = !item
      do
        break if (!equi.type.eq(|EQUI|))
        !equi = !equi.owner
      enddo
      if !this.equip.rtext.findFirst(!equi.string()).set() then
        !this.equip.val = !this.equip.rtext.findFirst(!equi.string())
        !this.collNozz()
        !this.nozz.val = !this.nozz.rtext.findFirst(!item.string())
      endif
    else
      !this.nozz.val = !this.nozz.rtext.findFirst(!item.string())
    endif
  endif
endmethod
```

As Part 1 but with modifications to methods .pick() and .identify()

Appendix B11 - Example c2ex6.pmlfrm – part 3

As Exercise 5 but with modifications to form definition and methods .pick() and .identify()

```
  ------------------------------
  --
  --  Modifications made to form
  --  definition and two methods:
  --
  ------------------------------

setup form !!c2ex6
  !this.formTitle       = |Equipment Checker|
  !this.initcall        = |!this.init()|
  !this.firstShownCall  = |!this.firstShown()|
  !this.killingCall     = |!this.close()|
  !this.newMenu(|pop1|)
  !this.pop1.add('CALLBACK', 'Go to Nozzle', '!!CE = !this.nozz.selection().dbref()', 'NOZZ')
  para .prompt text |Navigate : | width 46 lines 1
  frame .view panel at x 0.5 ymax.prompt wid 1 hei 1
    view .volumeView at x 0.5 ymax.prompt call '!!edgCntrl.canvasPick(' prompt .prompt VOLUME
      width 50 aspect 0.707
      border off
      shading on
      isometric 3
      inmode create _default    type |DES_NAVIGATE|
      inmode create _pick       type |DES_PICK|
    exit
  exit
  para     .title at xmax.view + 0.5 ymin.prompt text |Available Equipment|
  line     .titleLine at xmin.title ymax.title horiz wid 48
  combo    .equip at xmin.title + 0.5 ymax.titleLine + 0.2 call |!this.validate(| width 10
  list     .nozz  at xmin.equip ymax.equip call |!this.check()| width 45 length 5
  button   .site linklabel |Update| at xmax.equip + 0.5 ymin.equip call |!this.init()|
  button       .refreshNozz  at  xmax.nozz  -  size  ymax.nozz  tooltip  |Refresh  Nozzles|  call
|!this.checkNozz()| pixmap wid 16 hei 16
  button     .pick  tooltip  'Identify  Nozzle'  pixmap  at  xmin.refreshNozz  -  size  ymax.nozz  call
|!this.pick()| width 16 height 16
  textpane .atta |Equipment Attributes| at xmin.equip ymax.refreshNozz width 47 height 3.5
  button      .updateAtta  linklabel  |Update  Atts|  at  xmax.nozz  -  size  ymax.atta  +  0.2  call
|!this.updateAtt(2)| wid 7
  member .drawlist is REAL
exit

define method .pick()
  !packet = object EDGPACKET()
  !packet.elementPick(|Identify a nozzle - Escape to cancel|)
  !packet.description = |Identify Element|
  !packet.action = |!! c2ex6.identify(!this.return)|
  !!edgCntrl.add(!packet)
  !!edgCntrl.addView(!!c2ex6.volumeView)
endmethod

define method .identify(!pickInfo is ANY)
  !item = !pickInfo[1].item
  !!edgCntrl.remove(|Identify Element|)
  !!edgCntrl.removeView(!!c2ex6.volumeView)
  -- Loop to find the EQUI element
  if !item.type.eq(|NOZZ|) then
    !this.nozz.val = !this.nozz.rtext.findFirst(!item.string())
  else
    !this.pick()
  endif
endmethod
```

Appendix B12 - Example apppml.obj

PML addin file

```
name:         PMLTraining
title:        PML Training
showOnMenu:   FALSE
object:       appPML
```

apppml.obj

```
define object appPML
endobject

define method .modifyMenus()
 !this.utilitiesMenu()
endmethod

define method .modifyForm()
endmethod

define method .utilitiesMenu()
  !menu = object APPMENU('SYSUTIL')
  !menu.add('SEPARATOR')
  !menu.add('FORM', |Calculator|, |c2ex1|)
  !menu.add('MENU', |Equipment Checker|, 'EquipCheck')
  !!appMenuCntrl.addMenu(!menu, 'EQUI')

  !menu= object APPMENU('EquipCheck')
  !menu.add('FORM', |Exercise 4...|, |c2ex4|)
  !menu.add('FORM', |Exercise 5...|, |c2ex5|)
  !!appMenuCntrl.addMenu(!menu, 'EQUI')
endmethod
```